

Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

КУРСОВА РОБОТА
з дисципліни **"Об'єктно-орієнтоване програмування"**
на тему: «РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ ДЛЯ МОДЕЛЮВАННЯ
ОБ'ЄКТІВ "Симулятор спортзалу TORNADO GYM. Макрооб'єкти: Front desk &
Intake, Cardio area, Weights area. Мікрооб'єкти: Athlete, Power Lifter, Gym Boss"
З ВИКОРИСТАННЯМ UML ТА МОВИ ПРОГРАМУВАННЯ JAVA»

Виконав: студент 1-го курсу групи 2ПІ-256
спеціальності F2 Інженерія програмного
забезпечення
Ярослав СИЧ



Керівник доцент кафедри ПЗ
к.т.н., доц. Олександр РЕШЕТНІК

Кількість балів: 82
Оцінка: ЄКТС В

Члени комісії:

(підпис)

Решетнік О.О.

(підпис)

Кательніков Д.І.

(підпис)

(прізвище та ініціали)

м. Вінниця - 2026 рік

Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення
Рівень вищої освіти перший (бакалаврський)
Спеціальність F2 Інженерія програмного забезпечення
Освітньо-професійна програма «Інженерія програмного забезпечення»



ЗАТВЕРДЖУЮ

Завідувач кафедри ПЗ

д.т.н., проф., О.Н. Романюк

«10» лютого 2026 року

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ

на курсову роботу з дисципліни "Об'єктно-орієнтоване програмування"

Сичу Ярославу Олександровичу, студенту гр. 2ПІ-256

Тема роботи: РОЗРОБКА ПРОГРАМНОЇ СИСТЕМИ ДЛЯ МОДЕЛЮВАННЯ ОБ'ЄКТІВ "Симулятор спортзалу TORNADO GYM. Макрооб'єкти: Front desk & Intake, Cardio area, Weights area. Мікрооб'єкти: Athlete, Power Lifter, Gym Boss" З ВИКОРИСТАННЯМ UML ТА МОВИ ПРОГРАМУВАННЯ JAVA

Завдання: розробити програмну систему і пояснювальну записку до процесу розробки.

Вхідні дані до виконання проєкту:

Мова програмування: Java.

Середовище розробки: NetBeans IDE, IntelliJ IDEA, Eclipse або Visual Studio Code.

Графічний інтерфейс: платформа JavaFX, Swing або AWT.

Зображення мікрооб'єкта: мінімум три графічних примітива (один з яких - текст). Зображення кожного макрооб'єкту повинно складатись не менше чим з трьох графічних примітивів (лінія, прямокутник, коло, арка, полігон, текст, графічний образ та інших), при цьому обов'язково один з примітивів повинен бути текст. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання. Для класу

мікрооб'єкта слід реалізувати глибинне копіювання. Макрооб'єкти повинні мати можливість містити кількість мікрооб'єктів більше двох. Повинна бути реалізована ієрархія класів мікрооб'єктів, яка містить як мінімум три рівня наслідування, які можуть інстанціюватись і відображатись на екрані. Повинен бути реалізований наступний запит: знайти мікрооб'єкт з вказаними параметрами. Програма шукає об'єкт з введеними параметрами і видає на екран: його місце розташування, приналежність до макрооб'єктів (чи зазначає, що мікрооб'єкт не належить жодному макрооб'єкту). Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які належать вказаному макрооб'єкту. Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які не належать жодному макрооб'єкту. Повинен бути реалізований наступний запит: встановити критерій сортування мікрооб'єктів у запитах, які виводять переліки. Критеріїв сортування повинно бути не менше трьох, наприклад: Ім'я_студента, Середній_бал, Кількість_Оцінок_4_та_вище. Повинен бути реалізований автоматичний рух деяких мікрооб'єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою; 2) деякі мікрооб'єкти заходять в макрооб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Необхідно реалізувати щонайменше три запити на підрахунок мікрооб'єктів. Наприклад: а) кількість активних мікрооб'єктів; б) кількість об'єктів із рівнем життя понад 50%; в) кількість об'єктів, що розташовані у правій частині екрана тощо. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач. Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) на мінікарті повинна бути позначена видима область універсального об'єкта; 2) рух видимої області універсального об'єкта відповідно змінює мінікарту; 3) макрооб'єкти та мікрооб'єкти позначені на мінікарті; 4) рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті; 5) натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на екрані.

Зміст пояснювальної записки:

АНОТАЦІЯ

ЗМІСТ

ВСТУП

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ
ЗАВДАННЯ НА РОБОТУ

2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

4 КЕРІВНИЦТВО КОРИСТУВАЧА

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

ДОДАТОК А (обов'язковий) Лістинг програми

Дата видачі завдання: "11" лютого 2026 р.

Строк подання завершеної курсової роботи: 18 червня 2026 р.

Керівник _____

Signature
(підпис)

Олександр РЕШЕТНІК

Завдання отримав _____

Such
(підпис)

Ярослав СИЧ

АНОТАЦІЯ

Сич Я.О. Розробка програмної системи для моделювання об'єктів "Симулятор спортзалу Tornado GYM. Макрооб'єкти: Front desk & Intake, Cardio area, Weights area. Мікрооб'єкти: Athlete, Power Lifter, Gym Boss: курсова робота з дисципліни «Об'єктно-орієнтоване програмування»

У курсовій роботі здійснена розробка програмного застосунку для ОС Windows/Linux/Mac OS, розроблений на базі JavaFX. Програмний застосунок моделює функціонування об'єктів "атлети, зони спортзалу, спорядження" з використанням UML та мови програмування Java. Використано усі основні принципи об'єктно-орієнтованого програмування, а саме: абстракція, інкапсуляція, успадкування та поліморфізм. Досліджено спадну тенденцію зацікавленості молоді здоровим способом життя. Створено готовий програмний продукт.

Ключові слова: мікрооб'єкти, успадкування, макрооб'єкти, агрегація, універсальний об'єкт, композиції.

ЗМІСТ

ВСТУП	5
1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ	8
1.1 Предметна область	8
1.2 Розробка технічного завдання на роботу	11
1.3 Обґрунтування вибору мови програмування	15
1.4 Висновки	19
2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ	20
2.1 Модель графічного відображення	20
2.2 Графічні процедури підсистеми графічного відображення	26
3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ	28
3.1 Діаграма наслідування	28
3.2 Діаграма агрегації	30
3.3 Діаграма композиції	31
3.4 Діаграма асоціації	33
3.5 Діаграма кооперації	33
3.6 Діаграма послідовності	34
3.7 Діаграма стану	36
4 КЕРІВНИЦТВО КОРИСТУВАЧА	38
4.1 Головне вікно програми	38
4.2 Додавання нового атлета	39
4.3 Переміщення атлетів	41
4.4 Редагування атлета	41
4.5 Видалення та клонування атлетів	43
4.6 Сортування та статистика	43
4.7 Збереження та завантаження стану	44
4.8 Повідомлення про помилки та особливі ситуації	45

ВИСНОВКИ	46
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	47
ДОДАТОК А (обов'язковий) Лістинг програми	49

ВСТУП

Сучасний етап розвитку людської цивілізації характеризується докорінними змінами у способі життя людини, що зумовлені стрімким прогресом інформаційних технологій та автоматизацією виробничих процесів. Глобальна цифровізація, окрім очевидних переваг, принесла із собою низку викликів, серед яких чи не найголовнішим є гіподинамія — патологічний стан дефіциту рухової активності. Більшість професій сучасності перемістилися у віртуальну площину, що змушує молоде покоління та працездатне населення проводити значну частину часу в сидячому положенні перед моніторами. Це створює критичний дисбаланс між інтелектуальним навантаженням та фізичним станом організму.

Водночас у суспільній свідомості спостерігається трансформація ставлення до власного тіла та здоров'я. Фізична культура перестає сприйматися лише як професійна діяльність чи засіб підготовки до праці; вона перетворюється на невід'ємну частину гармонійного розвитку особистості, засіб соціалізації та важливе хобі. Проте на шляху до формування стабільної звички займатися спортом постає проблема мотивації. Традиційні методи залучення — лекції, заклики до здорового способу життя чи соціальна реклама — часто демонструють низьку ефективність у боротьбі з миттєвим задоволенням, яке пропонують цифрові розваги.

У цьому контексті виникає необхідність пошуку нових, інтерактивних підходів до стимулювання інтересу до спорту. Одним із таких інструментів є гейміфікація — використання ігрових механік, динаміки та естетики для залучення людей, стимулювання дій та розв'язання складних проблем[1, 2]. Створення програмних продуктів, які моделюють реальні життєві процеси, зокрема заняття у спортивному залі, дозволяє знизити «поріг входу» у спорт. Користувач, керуючи віртуальним персонажем, отримує безпечний простір для експериментів, вивчення основ тренувального процесу та, що найважливіше, наочну візуалізацію результатів від прикладених зусиль.

Створення гри-симулятора спортзалу є актуальним не лише як розважальний проєкт, а й як освітньо-мотиваційний інструмент. Вона дозволяє продемонструвати логічні зв'язки між дисципліною, регулярністю тренувань, правильним розподілом ресурсів (часу та енергії) та прогресом характеристик особистості. Таке віртуальне хобі може стати першим кроком до реальної трансформації способу життя, оскільки ігрові механізми (система досягнень, рівнів та візуального покращення персонажа) працюють за принципом позитивного підкріплення, який легко переноситься на реальну практику.

З технічної точки зору, актуальність дослідження зумовлена необхідністю застосування сучасних методологій розробки програмного забезпечення для створення складних інтерактивних систем. Вибір мови програмування Java для реалізації цього проєкту є стратегічно обґрунтованим. Java — це високорівнева, об'єктно-орієнтована мова програмування, яка забезпечує кросплатформеність, надійність та безпеку програмного продукту.

Проектування ігрового симулятора вимагає глибокого розуміння парадигми об'єктно-орієнтованого програмування (ООП). Структура спортивного залу ідеально декомпозиється на класи та об'єкти: сутності «Тренер», «Спортсмен», «Тренажер», «Інвентар», «План тренувань» мають власні атрибути та методи взаємодії. Використання Java дозволяє реалізувати складну логіку взаємодії цих об'єктів, забезпечуючи при цьому дотримання фундаментальних принципів програмування, таких як SOLID, DRY (Don't Repeat Yourself) та KISS (Keep It Simple, Stupid). Це гарантує створення чистого, тестованого та масштабованого коду, що є критично важливим для будь-якого професійного розробника.

Розробка графічного інтерфейсу та обробка ігрових подій у Java дозволяє занурити користувача у середовище, де кожен вибір має наслідки. Програмування логіки втоми, відновлення енергії та гіпертрофії віртуальних м'язів вимагає використання математичних моделей та алгоритмів, що робить цю курсову роботу комплексним дослідженням на стику психології мотивації, спортивної фізіології та комп'ютерних наук.

Мета курсової роботи полягає у теоретичному обґрунтуванні, проєктуванні та практичній реалізації інтерактивної гри-симулятора спортивного залу мовою програмування Java, яка через механізми гейміфікації сприятиме популяризації спорту та здорового способу життя серед користувачів.

Для досягнення поставленої мети необхідно вирішити наступні завдання:

1. Проаналізувати сучасні тенденції у сфері розробки ігрових симуляторів та методи гейміфікації як засобу стимулювання саморозвитку.
2. Дослідити предметну область (структуру тренувального процесу) для створення адекватної ігрової моделі.
3. Сформулювати вимоги до програмного продукту та розробити концепцію ігрових механік.
4. Спроектувати архітектуру застосунку, визначивши ключові класи, їхні властивості та методи взаємодії на основі принципів ООП.
5. Реалізувати логіку симулятора засобами мови Java, включаючи системи прогресії персонажа та управління ресурсами.
6. Розробити графічний користувацький інтерфейс, що забезпечує зручність та інтерактивність процесу.
7. Провести комплексне тестування розробленого програмного забезпечення на відповідність функціональним вимогам.

Об'єктом дослідження є процес створення ігрових симуляторів як інструментів моделювання соціально значущих активностей та засобів мотивації користувачів.

Предметом дослідження є методи, алгоритми та програмні засоби мови Java, а також принципи об'єктно-орієнтованого проєктування, що застосовуються для розробки десктопної гри-симулятора спортзалу Tornado Gym.

Саме тому в якості теми курсової роботи було обрано розробку програмної системи для моделювання об'єктів “Симулятор спортзалу TORNADO GYM. Макрооб'єкти: Front desk & Intake, Cardio area, Weights area. Мікрооб'єкти: Athlete, Power Lifter, Gym Boss”.

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ

1.1 Предметна область

Спортивний зал є складною соціальною системою, де одночасно взаємодіють різні категорії відвідувачів - від початківців до досвідчених атлетів. Кожна людина приходить до залу з власними цілями, рівнем підготовки та поведінковими особливостями. Саме це різноманіття робить фітнес-центр цікавим об'єктом для моделювання засобами об'єктно-орієнтованого програмування.

Сучасне суспільство стикається з проблемою зростаючої гіподинамії - хронічної нестачі фізичної активності. За даними Всесвітньої організації охорони здоров'я (ВООЗ, 2022)[3], понад 1,4 мільярда дорослих у всьому світі ведуть недостатньо активний спосіб життя, що збільшує ризик серцево-судинних захворювань, цукрового діабету 2-го типу та депресії. В Україні, за оцінками соціологічних досліджень, лише близько 20-25% молоді регулярно займаються фізичними вправами.

Симулятор «Tornado GYM» моделює динамічне середовище фітнес-центру з видом зверху. В ньому визначено трирівневу ієрархію мікрооб'єктів-відвідувачів та три функціональні зони-макрооб'єкти.

Ієрархія мікрооб'єктів-атлетів побудована за принципом успадкування:

Athlete (базовий клас).

Звичайний відвідувач спортзалу, початківець або людина, що займається підтримкою форми. Характеристики: помірна швидкість пересування, стандартна тривалість тренування, перевага кардіо-зони. Клас містить основні поля: ім'я (String name), рівень витривалості (double stamina), поточну зону (String currentArea), швидкість (double speed). Графічне представлення: кругла фігура жовтого кольору з підписом імені та смужкою витривалості. Зображення доповнюється тінню у вигляді еліпса.

Power Lifter (нащадок Athlete).

Досвідчений спортсмен, що спеціалізується на силових тренуваннях. Успадковує всі поля та методи класу Athlete і розширює їх: додається поле `int maxWeight` (максимальна робоча вага), `double recoveryRate` (швидкість відновлення). Поведінка: вища витривалість, сповільнений рух (велика «маса»), виражена перевага зони `Weights area`. **Power Lifter** перевизначає метод `train()` батьківського класу, викликаючи `super.train()` для базової логіки та додаючи силові вправи. Візуально відрізняється від Athlete більшими розмірами фігури синього кольору.

Gym Boss (нащадок Power Lifter).

Елітний відвідувач - авторитет спортзалу, найвищий рівень ієрархії. Успадковує поля **Power Lifter** та додає: `int influenceRadius` (радіус впливу на оточуючих), `boolean isMotivating` (ознака активної мотивації). **Gym Boss** перевизначає метод `interact()` та викликає `super.interact()` з **Power Lifter**. Унікальна здатність: при знаходженні в зоні впливу інших атлетів він підвищує їх витривалість (`motivate()`). Займає тренажери позачергово. Графічне представлення: фігура червоного кольору з золотистим ореолом та короною-символом.

Ієрархія успадкування ілюструє ключові принципи ООП. Кожен наступний рівень розширює попередній, не повторюючи код (принцип DRY). Усі три класи можуть бути відображені на екрані та взаємодіяти між собою - це демонструє поліморфізм: масив `List<Athlete>` може містити об'єкти всіх трьох типів, і кожен виконає власну реалізацію методу `train()`.

Різниця між класами по наслідуванню наведена в таблиці 1.1.

Таблиця 1.1 - Порівняння характеристик класів мікрооб'єктів

Характеристика	Athlete	Power Lifter	Gym Boss
Рівень ієрархії	1 (базовий)	2 (нащадок Athlete)	3 (нащадок Power Lifter)
Швидкість руху	Середня	Низька (важкий)	Низька + пріоритет
Витривалість	Стандартна	Висока	Максимальна
Пріоритетна зона	Cardio area	Weights area	Weights area
Особлива здатність	Відсутня	Силові вправи	Мотивація сусідів (motivate())
Колір фігури	Жовтий	Синій	Червоний + золотий ореол
Додаткові поля	name, stamina, speed, currentZone	+ maxWeight, recoveryRate	+ influenceRadius, isMotivating

Макрооб'єкти-зони спортзалу є контейнерами для мікрооб'єктів. У симуляторі «Tornado GYM» визначено три зони:

Front desk & Intake - вхідна зона та ресепшн.

Тут атлети з'являються на початку симуляції та проходять реєстрацію. Зона слугує точкою входу: новостворені об'єкти атлетів з'являються саме тут та звідси прямують до тренувальних зон. Це відповідає реальному процесу відвідування спортзалу і додає автентичності симуляції.

Cardio area - кардіо-зона.

Містить бігові доріжки та велотренажери. Орієнтована переважно на атлетів базового рівня (Athlete). Атлети у цій зоні виконують аеробні вправи, поступово підвищуючи свої показники витривалості.

Weights area - зона важкої атлетики. Містить силові тренажери та вільні ваги. Пріоритетна зона для Power Lifter та Gym Boss. Атлети взаємодіють між собою: присутність Gym Boss підвищує інтенсивність тренування сусідніх атлетів.

Усе моделювання відбувається у межах універсального об'єкта GymWorld - контейнера найвищого рівня, який об'єднує всі зони та всіх атлетів. Вікно програми відображає лише частину простору GymWorld (не більше 25% від загального розміру), що створює відчуття великого, живого спортзалу.

1.2 Розробка технічного завдання на роботу

В програмі на мові програмування Java повинно бути створено застосунок з графічним інтерфейсом. Клас мікрооб'єкта повинен містити не менше 4 елементів змінних і не менше 4 методів (окрім геттерів та сеттерів). Як мінімум одна змінна повинна бути типу `int`, одна - типу `double` і як мінімум одна - рядок. Зображення мікрооб'єкта: мінімум три графічних примітива, один з яких - текст (навіть в неактивному стані). Білий (невидимий) графічний примітив не рахується. Додати до класу мікрооб'єкта в якості елемента посилальний тип, щоб зробити необхідним глибинне копіювання. Для класу мікрооб'єкта слід реалізувати глибинне копіювання. Зображення мікрооб'єкта повинно містити графічні примітиви, які показують стан елемента посилального типу, який був доданий до класу мікрооб'єкта, щоб зробити необхідним глибинне копіювання при клонуванні. При натискуванні лівої кнопки миші на мікрооб'єкті він повинен ставати активним/неактивним. В програмі повинна бути реалізована активація декількох об'єктів одночасно. Зображення активного мікрооб'єкта повинно відрізнятись від зображення неактивного хоча б одним графічним примітивом. Інтерфейс програми повинен сигналізувати про те, який саме мікрооб'єкт активував користувач. По вибору студента інформація про активний мікрооб'єкт/мікрооб'єкти може виводитись у рядку статусу або робочій області програми. Якщо користувач активував декілька мікрооб'єктів, то інформація про це також має якимось чином відображатися. При натискуванні клавіш-стрілок активні об'єкти/об'єкт повинні

рухатись у вікні програми. При натискуванні клавіши Delete активні об'єкти повинні знищуватись. Якщо активного об'єкта нема - клавіша, то клавіша Delete ігнорується. Клавіша Esc повинна відмінати активацію об'єкта. По певній комбінації клавіш (наприклад Ctrl-C) повинно бути реалізоване копіювання активного мікрооб'єкта. При натискуванні правої кнопки миші на мікрооб'єкті повинно з'являтися діалогове вікно, яке дозволяє змінювати параметри вказаного мікрооб'єкта. Додати до проекту три або більше макрооб'єкта. Макрооб'єкти повинні мати можливість містити більше двох мікрооб'єктів одночасно. Додати команди рисування макрооб'єктів в програму. Не обов'язково мати можливості додавати/видаляти макрооб'єкти в програму. Цілком достатньо, якщо програма буде мати деяку кількість макрооб'єктів згенеровану при запуску програми і яка буде незмінною протягом роботи програми. Макрооб'єкти повинні мати певні позиції в універсальному об'єкті і повинні відображатись на екрані. Позиції макрооб'єктів всередині універсального об'єкта можуть залишатись незмінними протягом роботи програми. В універсальному об'єкті повинно міститись як мінімум один екземпляр кожного оголошеного в темі макрооб'єкта. Зображення кожного макрооб'єкту повинно складатись не менше чим з трьох графічних примітивів (лінія, прямокутник, коло, арка, полігон, текст, графічний образ та інших), при цьому обов'язково один з примітивів повинен бути текст. Білий (=невидимий) графічний примітив не рахується. Мікрооб'єкти можуть або належати одному або більшій кількості макрооб'єктів або не належати жодному. Глядачу має бути зрозуміло в чому полягає приналежність мікрооб'єкта макрооб'єкту, тобто чим вигляд (або поведінка) такого мікрооб'єкта відрізняється від поведінки мікрооб'єкта, який не належить жодному макрооб'єкту. Повинна обов'язково бути реалізована можливість вилучити мікрооб'єкт з всіх макрооб'єктів, в результаті чого він не буде належати жодному макрооб'єкту. Повинна обов'язково бути реалізована можливість зробити так, щоб активний мікрооб'єкт став належати одному з наявних макрооб'єктів. В зображенні макрооб'єкта обов'язково повинно виводитись кількість мікрооб'єктів, які йому належать. Повина бути побудована ієрархія класів мікрооб'єктів, яка містить як

мінімум три рівня наслідування, які можуть інстанціюватись і зображатись на екрані. Зображення кожного рівня має відрізнятись від всіх інших хоча б одним графічним примітивом. В побудованій ієрархії класів повинні бути продемонстровані: а) виклик конструктора базового класу; б) виклик метода базового класу; в) використання динамічного поліморфізму; г) використання статичного поліморфізму. Повинно бути продемонстровано використання ключового слова `instanceof`. Повинен бути реалізований автоматичний рух деяких мікрооб'єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою (це має бути помітно візуально в тому сенсі, що або щось змінюється у їх зображенні або у характері їх руху); 2) деякі мікрооб'єкти заходять в макрооб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Під характером руху мається на увазі не просто траєкторія, а те, в який спосіб вони взаємодіють з макрооб'єктами та іншими мікрооб'єктами. У вікні програми відображається лише певна частина ігрового світу, яка не повинна перевищувати 25% його загального розміру. Реалізувати можливість змінювати частину ігрового світу, яку спостерігає користувач (будь яким способом). Повинна бути реалізована мінікарта, яка дозволяє прискорену навігацію: 1) на мінікарті повинна бути позначена видима область універсального об'єкта; 2) рух видимої області універсального об'єкта відповідно змінює мінікарту; 3) макрооб'єкти та мікрооб'єкти позначені на мінікарті; 4) рух та взаємодія мікрооб'єктів призводить до відповідних змін на мінікарті; 5) натискування лівої кнопки миші на мінікарті призводить до переміщення видимої області універсального об'єкта. Повинна бути запрограмована серіалізація/де-серіалізація всіх об'єктів у текстовий/бінарний/XML файл. Серіалізація/де-серіалізація обов'язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об'єктів, але й про їх позицію на карті. При серіалізації/де-серіалізації обов'язково повинні використовуватись діалогові вікна, щоб запитати у користувача ім'я файлу та його розташування на диску (наприклад можна використовувати системне діалогове вікно `FileChooser`). Продемонструвати використання лямбда-виразу.

Продемонструвати використання операція з потоком Stream. При натискуванні клавіші Insert повинно з'являться діалогове вікно, яке повинно визначати параметри створюваного мікрооб'єкта. В нього повинна бути присутня можливість обирати до якого з класів-нащадків у ієрархії наслідування він належить. Крім керуючого елемента Button у діалоговому вікні також повинні бути використані як мінімум три з наступного списку: TextField, CheckBox, {ComboBox || ListView || ChoiceBox}, RadioButton.

Повинен бути реалізований наступний запит: знайти мікрооб'єкт з вказаними користувачем параметрами, в результаті якого програма шукає мікрооб'єкт і видає на екран: його місце розташування, приналежність цього мікрооб'єкта до макрооб'єктів (чи зазначає, що мікрооб'єкт не належить жодному макрооб'єкту). Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які належать вказаному макрооб'єкту. Повинен бути реалізований наступний запит: видати перелік мікрооб'єктів, які не належать жодному макрооб'єкту. Повинен бути реалізований наступний запит: встановити критерій сортування мікрооб'єктів у запитах, які виводять переліки. Критеріїв сортування повинно бути не менше трьох, наприклад: Ім'я_студента, Середній_бал, Кількість_Оцінок_4_та_вище.

Повинні бути реалізовані як мінімум три запити по підрахуванню мікрооб'єктів. Наприклад а) кількість активних мікрооб'єктів, кількість мікрооб'єктів з рівнем життя більше половини, кількість мікрооб'єктів, які знаходяться в правій половині екрану тощо.

Для нормального виконання програми необхідно наступне апаратне забезпечення:

1. Комп'ютер серії IBM PC з частотою 2200 МГц і вище.
2. Операційні системи: Windows Vista/7/10; Linux; Mac OS.
3. 8 Gb оперативної пам'яті DDR4.
4. Графічний адаптер SVGA (Super Video Graphic Adapter).
5. Відео-карта об'ємом пам'яті не менше 2 Gb.
6. Клавіатура IBM, миша.

Розмір дискового простору, який потрібен для коректної роботи програми: 25 Мб.

Розмір оперативної пам'яті, що займає програма: 270 Мб.

1.3 Обґрунтування вибору мови програмування

Для виконання поставленого завдання було обрано кросплатформенну мову програмування Java у поєднанні з платформою JavaFX та середовищем розробки IntelliJ IDEA[4, 5]. Основними конкурентами при виборі були мови C++ та C#. Розглянемо, чому для реалізації симулятора «Tornado GYM» Java є оптимальним вибором.

Повноцінна підтримка парадигми ООП. Java є однією з небагатьох мов, де об'єктно-орієнтована парадигма є не лише підтримуваною, а й основоположною. Усі конструкції мови побудовані навколо класів та об'єктів. Це дозволяє природно реалізувати трирівневу ієрархію атлетів (Athlete → Power Lifter → Gym Boss), поліморфну поведінку та інкапсуляцію без архітектурних компромісів.

Кросплатформеність (WORA). Завдяки принципу «write once, run anywhere» Java-програма виконується на будь-якій операційній системі, де встановлено JVM. Це підвищує доступність «Tornado GYM» для потенційних користувачів незалежно від їхньої платформи, тоді як C++, наприклад, потребує перекомпіляції під кожну платформу.

JavaFX як сучасний GUI-фреймворк. Платформа JavaFX надає необхідні засоби[6, 7] для симулятора: Canvas API для довільного малювання об'єктів на ігровому полі, AnimationTimer та Timeline для реалізації ігрового циклу з плавною анімацією, підтримку подій клавіатури та миші. На відміну від застарілого Swing, JavaFX використовує апаратне прискорення графіки через OpenGL/DirectX.

Вбудована підтримка серіалізації. Стандартна бібліотека Java містить механізм серіалізації об'єктів (інтерфейс Serializable)[8, 9], а також засоби роботи з XML та JSON. Це спрощує реалізацію збереження та завантаження повного стану симуляції «Tornado GYM» без підключення зовнішніх бібліотек.

Перелік властивостей, характерних лише Java у порівнянні з конкурентами:

1. Java є повністю кросплатформенною мовою загального застосування. Програми, написані на ній, працюють буквально на всьому - від суперкомп'ютерів до мікрочіпів, тоді як C++ з самого початку орієнтований на компіляцію конкретною платформою.
2. Java надає широкі можливості для багатопоточності, що дозволяє ігровому циклу «Tornado GYM» та підсистемі серіалізації працювати паралельно.
3. Java має повну зворотну сумісність з попередніми версіями - сьогодні чудово працюватимуть проекти, створені декілька років тому.
4. Концепція ООП набагато краще реалізована в Java, ніж в інших мовах програмування. Java бере на себе всю низькорівневу роботу з пам'яттю: видаленням непотрібних об'єктів займається вбудований Garbage Collector (збирач сміття).

Перелік недоліків Java у порівнянні з C++ і C#:

1. Java повинна розплачуватися за кросплатформеність швидкодією: код, написаний на C++, виконується швидше та потребує менше ресурсів комп'ютера.
2. У Java відсутня перевантаження стандартних операторів, яка притаманна C#, що в деяких нестандартних ситуаціях робить код менш зрозумілим.
3. Java не надає можливостей для створення генераторів, які є в C#, JavaScript та Python.

Таблиця 1.2 - Порівняння мов програмування

Особливості	C++	C#	Java
Парадигма	Мультипарадигмальна (включає ООП та процедурне програмування), що іноді ускладнює архітектуру.	ООП та компонентно-орієнтована, сильно інтегрована з екосистемою Microsoft.	Еталонне ООП, ідеально структуроване для створення масштабованих/легко підтримуваних рішень.

Продовження таблиці 1.2

Особливості	C++	C#	Java
Рік випуску	1983 (має великий багаж застарілих "legacy" підходів).	2000 (створювалась як відповідь Microsoft на успіх Java).	1995 (перевірена часом, має найстабільніший шлях еволюції).
Механізм виконання	Пряма компіляція в залежний від машини код.	Компіляція в проміжний код (IL), орієнтована переважно на середовище .NET.	Байт-код + JVM, що забезпечує унікальну гнучкість, незалежність від заліза та динамічну оптимізацію.
Сфера застосування	Вузькоспеціалізовані задачі: геймдев, драйвери, високонавантажені системні модулі.	Розробка корпоративних рішень переважно під Windows, ігри (Unity).	Універсальність: беззаперечний лідер у Enterprise-секторі, хмарних сервісах, банківських системах та розробці під Android.
Синтаксис та читабельність	Перевантажений і важкий для сприйняття, високий поріг входження.	Сучасний, але місцями переобтяжений "синтаксичним цукром".	Лаконічний, строгий та інтуїтивно зрозумілий, що робить код максимально читабельним.
Кросплатформеність	Низька. Вимагає окремої перекомпіляції під кожен операційну систему.	Добра, але історично найкраще працює саме в середовищі Windows.	Абсолютна ("Write Once, Run Anywhere"). Працює скрізь, де встановлена Java Virtual Machine.

Продовження таблиці 1.2

Особливості	C++	C#	Java
Кросплатформеність	Низька. Вимагає окремої перекомпіляції під кожен операційну систему.	Добра, але історично найкраще працює саме в середовищі Windows.	Абсолютна ("Write Once, Run Anywhere"). Працює скрізь, де встановлена Java Virtual Machine.
Продуктивність	Максимальна, але ціною підвищеної складності розробки та ризику помилок.	Висока, оптимізована під інфраструктуру .NET.	Висока та стабільна. Сучасні JIT-компілятори нівелюють різницю в швидкості, забезпечуючи ідеальний баланс продуктивності та надійності.
Безпека та пам'ять	Ручне управління пам'яттю (постійний ризик витоків пам'яті та критичних вразливостей).	Автоматичне збирання сміття, надійна архітектура в рамках .NET.	Безпрецедентний рівень безпеки. Ізольоване виконання в JVM та передові алгоритми Garbage Collector унеможливають більшість фатальних помилок.
Екосистема (Розширюваність)	Фрагментована, відсутність єдиного стандарту для управління пакетами.	Потужна, але сильно централізована навколо інструментів та платних рішень Microsoft.	Найбільша у світі Open Source спільнота. Величезний вибір безкоштовних бібліотек та еталонних фреймворків, що охоплюють як десктопні, так і серверні рішення[10].

Таким чином, беручи до уваги результати багатокритеріального порівняння мов об'єктно-орієнтованого програмування(див. таблицю 1.2), можна зробити

висновок, що для реалізації поставленого завдання оптимальною є мова програмування Java у поєднанні з платформою JavaFX та середовищем IntelliJ IDEA.

1.4 Висновки

Отже, у першому розділі було виконано комплексний аналіз предметної галузі та обґрунтування завдання на курсову роботу.

Визначено предметну область симулятора «Tornado GYM» - динамічне середовище спортивного залу з видом зверху. Описано трирівневу ієрархію мікрооб'єктів-атлетів: базовий клас Athlete (звичайний відвідувач), клас Power Lifter (нащадок Athlete, силовий спортсмен) та клас Gym Boss (нащадок Power Lifter, елітний авторитет залу). Наведено порівняльну таблицю цих класів, яка ілюструє принцип успадкування та розширення функціональності: кожен рівень ієрархії додає нові поля та перевизначає методи батьківського класу, демонструючи поліморфізм. Gym Boss, зокрема, має унікальну здатність мотивувати сусідніх атлетів через метод `motivate()`, що є прикладом динамічного поліморфізму.

Описано три макрооб'єкти-зони: Front desk & Intake (вхідна зона), Cardio area (кардіо-зона для Athlete) та Weights area (силова зона для Power Lifter та Gym Boss). Усі вони функціонують у межах контейнера GymWorld.

Розроблено технічне завдання на роботу: визначено функціональні та нефункціональні вимоги до програми, апаратні та програмні вимоги до системи. Обґрунтовано вибір мови програмування Java та платформи JavaFX за такими критеріями: повноцінна підтримка ООП, кросплатформеність, сучасний GPU-прискорений GUI, вбудована серіалізація та промислова затребуваність. Проведено порівняльний аналіз Java, C++ та C# у таблиці 1.2, що підтверджує оптимальність обраної мови для реалізації симулятора.

2 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

2.1 Модель графічного відображення

Кожен мікрооб'єкт-атлет складається з кількох графічних елементів JavaFX, що утворюють єдину візуальну групу (Group). Усі елементи позиціонуються відносно двох базових координат X та Y, які є приватними полями класу Athlete. Значення зміщень визначені безпосередньо в коді методу `initVisuals()`.

У симуляторі «Tornado GYM» визначено три типи мікрооб'єктів-атлетів, що утворюють трирівневу ієрархію успадкування. Кожен тип має власне графічне представлення.

Athlete (базовий клас) — звичайний відвідувач спортзалу. Зображений у вигляді стрункої фігури в чорній футболці з логотипом «Tornado GYM» та чорних шортах. Графічне представлення включає: PNG-зображення персонажа (`imageView`), текстовий блок з іменем (`nameText`) та назвою екіпірування (`equipmentText`), смужку витривалості жовтого кольору (`energyBar`) та райдужну рамку активації (`activeBorder`). Зображення мікрооб'єкта Athlete наведено на рисунку 2.1, а.

Power Lifter (нащадок Athlete) — досвідчений силовий атлет. Зображений масивною фігурою у синьому важкоатлетичному комбінезоні з логотипом «Tornado GYM». Графічне представлення ідентичне за структурою до Athlete, проте використовує інше PNG-зображення персонажа збільшеного масштабу. Зображення мікрооб'єкта Power Lifter наведено на рисунку 2.1, б.

Gym Boss (нащадок Power Lifter) — елітний авторитет залу, найвищий рівень ієрархії. Зображений масивною фігурою у червоному спортивному костюмі. Графічне представлення ідентичне за структурою, проте використовує власне PNG-зображення. Зображення мікрооб'єкта Gym Boss наведено на рисунку 2.1, в.



Рисунок 2.1 - Зображення мікрооб'єктів у програмі: а) Athlete; б) Power Lifter; в) Gym Boss

Координати всіх складових елементів мікрооб'єкта Athlete розраховуються відносно базових координат X та Y , що наочно показано на рисунку 2.1. Відповідні значення координат кожного елемента наведено у таблиці 2.1.

Таблиця 2.1 - Координати елементів мікрооб'єкта Athlete відносно базової точки (X , Y)

Елемент	x	y	$\text{width} \times \text{height}$
textBg (Rectangle)	$X + 2$	$Y + 2$	156×40
nameText (Text)	$X + 5$	$Y + 18$	—
equipmentText (Text)	$X + 5$	$Y + 33$	—
energyBarBg (Rectangle)	$X + 5$	$Y + 40$	150×8

Продовження таблиці 2.1

Елемент	x	y	width × height
energyBar (Rectangle)	$X + 5$	$Y + 40$	$150 \times (\text{energyLvl}/100) \times 8$
zoneEmojiView (ImageView)	$X + 138$	$Y + 4$	22×22
imageView (ImageView)	$X + 10$	$Y + 50$	140×210
activeBorder (Rectangle)	$X + 10$	$Y + 50$	140×210

Кожен макрооб'єкт-зона є контейнером для мікрооб'єктів та складається з кількох графічних елементів JavaFX, що утворюють єдину групу (Group). Усі елементи позиціонуються відносно базових координат X, Y та параметрів width і height зони. У симуляторі «Tornado GYM» визначено три зони-макрооб'єкти.

Front desk & Intake — вхідна зона та ресепшн спортзалу. Являє собою стійку адміністратора з підсвіченим логотипом «Tornado GYM», полицями з харчовими добавками та інформаційними моніторами. Макрооб'єкт включає: PNG-зображення рецепції (width = 600, height = 280), тематичне емодзі-іконку перед назвою зони, текстову мітку з назвою зони та лічильник атлетів. Зображення макрооб'єкта Front desk & Intake наведено на рисунку 2.2.

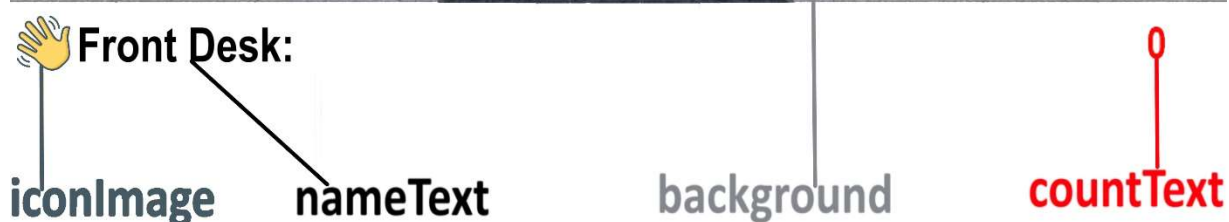


Рисунок 2.2 - Зображення макрооб'єкта Front desk & Intake

Cardio area — кардіо-зона. Містить ряди бігових доріжок з сенсорними моніторами, еліптичні тренажери та велотренажери з логотипом «Tornado GYM». Макрооб'єкт включає: PNG-зображення зони (width = 480, height = 320), тематичне емодзі-іконку перед назвою зони, текстову мітку з назвою та лічильник атлетів. Зображення макрооб'єкта Cardio area наведено на рисунку 2.3.

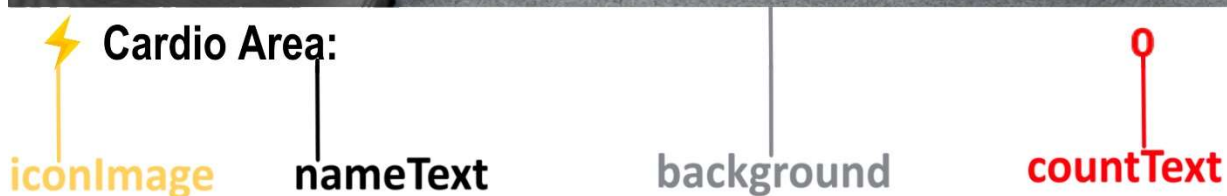
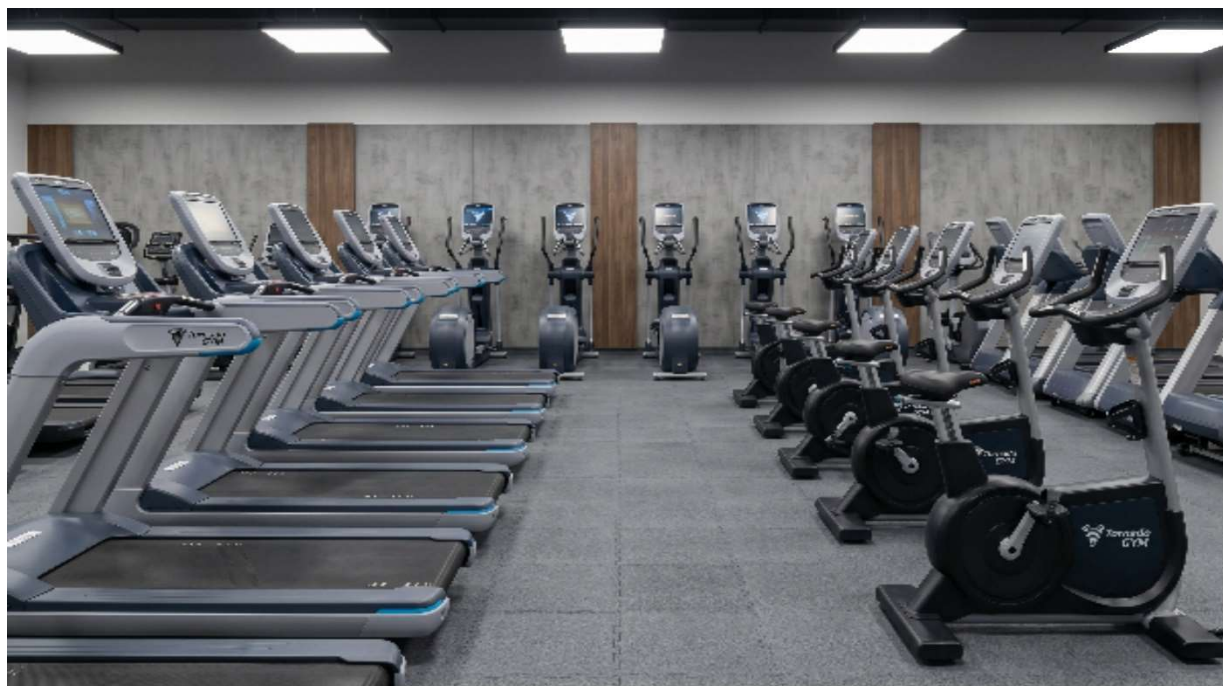


Рисунок 2.3 - Зображення макрооб'єкта Cardio area

Weights area — зона важкої атлетики. Містить силові стійки зі штангами, гантелями та дисками з логотипом «Tornado GYM», стійки з гантелями різної ваги та похилі лави. Макрооб'єкт включає: PNG-зображення зони (width = 480, height = 320), тематичне емодзі-іконку перед назвою зони, текстову мітку з назвою та лічильник атлетів. Зображення макрооб'єкта Weights area наведено на рисунку 2.4.

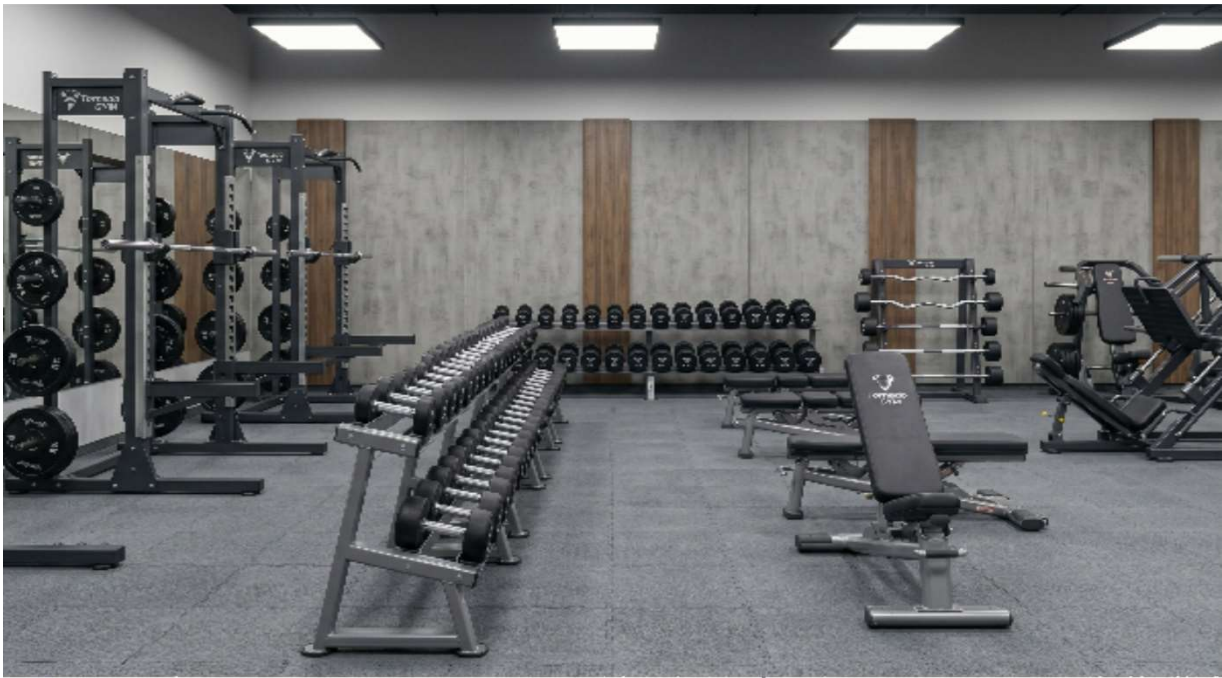


Рисунок 2.4 - Зображення макрооб’єкта Weights area

Координати всіх складових елементів макрооб’єкта TrainingZone розраховуються відносно базових координат X, Y та параметрів width і height зони, що наочно показано на рисунках 2.2-2.4. Відповідні значення координат кожного елемента наведено у таблиці 2.2.

Таблиця 2.2 - Координати елементів макрооб’єкта TrainingZone відносно базової точки (X, Y)

Елемент	x	y
background (ImageView)	X	Y
iconImage (ImageView)	X + 5	Y + height + 5
nameText (Text)	X + 35	Y + height + 23
countText (Text)	X + width – 40	Y + height + 23

Усі макрооб'єкти в своєму зображенні динамічно відображають кількість мікрооб'єктів, що знаходяться в них у конкретний момент часу симуляції. Загальний контейнер GymWorld об'єднує всі три зони та всіх атлетів, а вікно програми відображає лише частину його простору, що створює відчуття великого живого спортзалу.

2.2 Графічні процедури підсистеми графічного відображення

У програмі графічне відображення мікро- та макрооб'єктів реалізовано засобами бібліотеки JavaFX[4, 6]. Нижче наведено перелік ключових графічних примітивів і методів, що були задіяні в процесі розробки.

`ImageView(Image image)` — елемент-контейнер для відображення растрових зображень на сцені JavaFX, що є похідним від класу `Node`. Задіяний для виведення PNG-спрайтів персонажів та ігрових зон.

`Image(String string)` — виконує завантаження растрового зображення за вказаним шляхом у файловій системі або з ресурсного каталогу програми (`/resources/images/`).

`Label(String string)` — текстовий елемент інтерфейсу з визначеним вмістом. Застосовується для підпису імені атлета над його спрайтом, а також для відображення назви зони та поточної кількості атлетів у макрооб'єкті.

`Rectangle(double width, double height)` — прямокутна фігура-нащадок класу `Shape` із заданими габаритами. Використовується при побудові індикатора витривалості (`stamina bar`): статичний сірий прямокутник слугує фоном, а поверх нього розміщено динамічний прямокутник, ширина якого змінюється пропорційно до поточного значення поля `stamina`.

`setEffect(Effect value)` — метод встановлення візуального ефекту для будь-якого вузла сцени. У проєкті застосовується для накладання ефекту `GaussianBlur` на еліпс-тінь персонажів, що надає зображенню реалістичності.

`setRotate(double value)` — задає кут повороту елемента відносно його центральної осі. Використовується для легкого нахилу тіні під атлетами з метою створення ілюзії просторової глибини.

`setOpacity(double value)` — регулює рівень прозорості вузла в діапазоні 0.0–1.0. Застосовується для тіней (значення у межах 0.35–0.5) та під час анімації появи й зникнення об'єктів на сцені.

`Group(Node... children)` — контейнер, що об'єднує довільну кількість вузлів `Node` для їх колективного розміщення та керування на головному екрані. Кожен мікро- та макрооб'єкт інкапсульовано у власну групу, що спрощує їх переміщення і контроль за z-порядком відображення.

`setOnMouseClicked(EventHandler<MouseEvent> value)` — реєструє обробник події кліку на вузол. Відповідає за перемикання атлета в режим ручного керування: натискання лівої кнопки миші на зображенні персонажа активує або деактивує цей режим.

`AnimationTimer()` — основа ігрового циклу, що містить методи `start()`, `stop()` та `handle(long now)`, який викликається безперервно на кожному кадрі. Саме через `AnimationTimer` реалізовано ігровий цикл «Tornado GYM»: у методі `handle()` щокадрово перераховуються позиції атлетів, оновлюється стан індикаторів витривалості, перевіряються колізії з межами зон та між об'єктами, після чого сцена перемальовується.

`Timeline` та `KeyFrame` — інструменти для побудови плавних покадрових анімацій окремих подій: появи нового персонажа (ефект `fade-in`), активації здатності `motivate()` у `Gym Boss` (золотистий спалах навколо персонажа) та переходу атлета між зонами.

`Media(String url)` та `MediaPlayer(Media media)` — забезпечують завантаження та відтворення аудіофайлів формату `mp3` у середовищі програми. Звуковий супровід охоплює фонову музику тренажерного залу та звукові ефекти, що супроводжують взаємодію атлетів з тренажерами.

3 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

Діаграма класів (англ. class diagram) — статичне представлення структури моделі[11]. Вона відображає статичні (декларативні) елементи, такі як: класи, типи даних, їх зміст та відношення. У даному розділі розглянуто основні типи UML-діаграм, які відображають архітектуру симулятора спортзалу «Tornado GYM»: ієрархію мікрооб'єктів, зв'язки між класами, послідовність взаємодій та поведінку об'єктів у різних станах.

3.1 Діаграма наслідування

Наслідування — це один із базових принципів об'єктно-орієнтованого програмування, який дозволяє створювати нові класи на основі вже існуючих, успадковуючи їхні властивості та методи. Це дозволяє уникнути дублювання коду та відповідає принципу DRY (Don't Repeat Yourself)[12, 13]. У розробленій програмі реалізовано ієрархію мікрооб'єктів (рисунок 3.1).

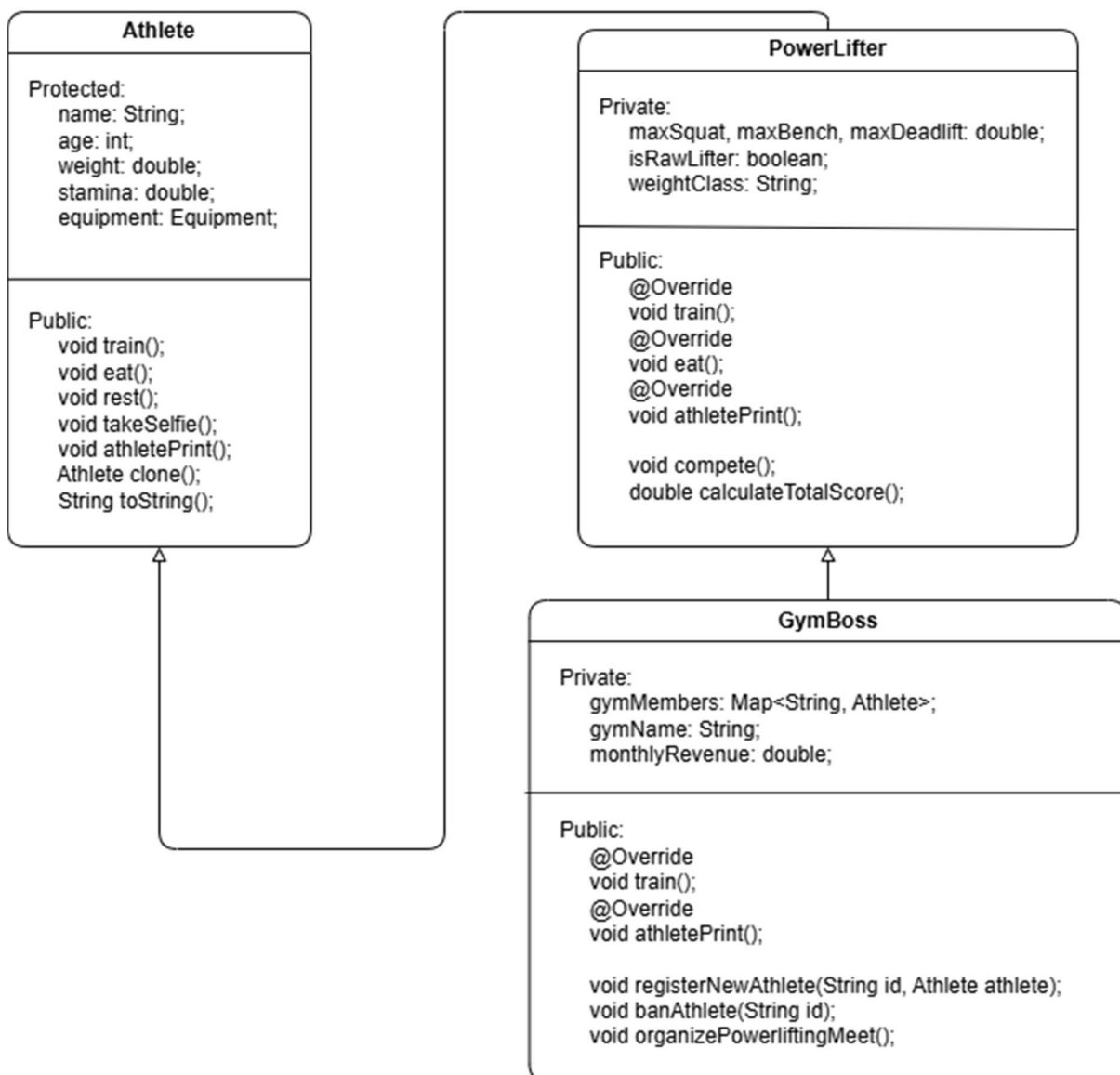


Рисунок 3.1 - Діаграма наслідування

Базовим класом виступає **Athlete** (Атлет), який містить спільні для всіх відвідувачів спортзалу поля: **name** (ім'я), **age** (вік), **weight** (вага), **stamina** (витривалість) та посилання на об'єкт екіпірування **equipment**. Серед основних методів базового класу присутні: **train()**, **eat()**, **rest()**, **takeSelfie()**, **athletePrint()**.

Клас-нащадок **PowerLifter** (Паверліфтер) розширює функціонал базового класу, додаючи специфічні для силового спорту поля (**maxSquat**, **maxBench**, **maxDeadlift**, **isRawLifter**, **weightClass**) та методи (**compete()**, **calculateTotalScore()**). Він також перевизначає методи **train()**, **eat()** та **athletePrint()** для реалізації поліморфізму.

Ще одним класом в ієрархії є GymBoss (Власник залу). Важливо зазначити, що у структурі даної програмної системи GymBoss виступає саме як повноцінний мікрооб'єкт (персонаж на карті), а не як макрооб'єкт чи системний менеджер. Він містить унікальні поля (gymMembers, gymName, monthlyRevenue) та специфічні методи керування залом (registerNewAthlete(), organizePowerliftingMeet()), при цьому поліморфно перевизначаючи базові методи тренування.

3.2 Діаграма агрегації

Агрегація відображає відношення "частина-ціле", де частина може існувати незалежно від цілого. У разі знищення об'єкта-контейнера, об'єкт-елемент продовжує існувати в системі. У симуляторі «Tornado GYM» агрегація продемонстрована зв'язком між класами Athlete та Equipment (Екіпірування) (рисунок 3.2).

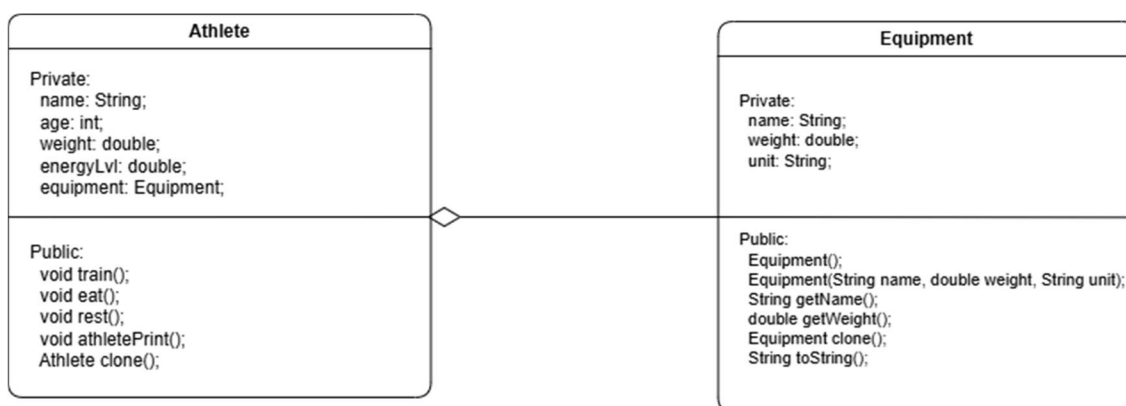


Рисунок 3.2 - Діаграми агрегації

Кожен мікрооб'єкт атлета може мати певне екіпірування (наприклад, пояс для важкої атлетики або фітнес-резинку), посилання на яке зберігається у полі `equipment`. Проте об'єкт `Equipment` є повністю самостійним: якщо об'єкт `Athlete` буде видалено з пам'яті (наприклад, персонаж покидає спортзал), саме екіпірування все ще може існувати або бути переданим іншому мікрооб'єкту.

3.3 Діаграма композиції

Композиція — це більш жорсткий варіант агрегації, при якому життєвий цикл частин повністю залежить від життєвого циклу контейнера. Об'єкт-частина не може існувати без об'єкта-цілого. У програмі композиція яскраво виражена при створенні графічного інтерфейсу тренувальних зон. Клас `TrainingZone` (Тренувальна зона) складається з низки графічних примітивів: `background` та `iconImage` (типу `ImageView`), `nameText` та `countText` (типу `Text`), `border` (типу `Rectangle`) та `visualGroup` (типу `Group`) (рисунок 3.3).

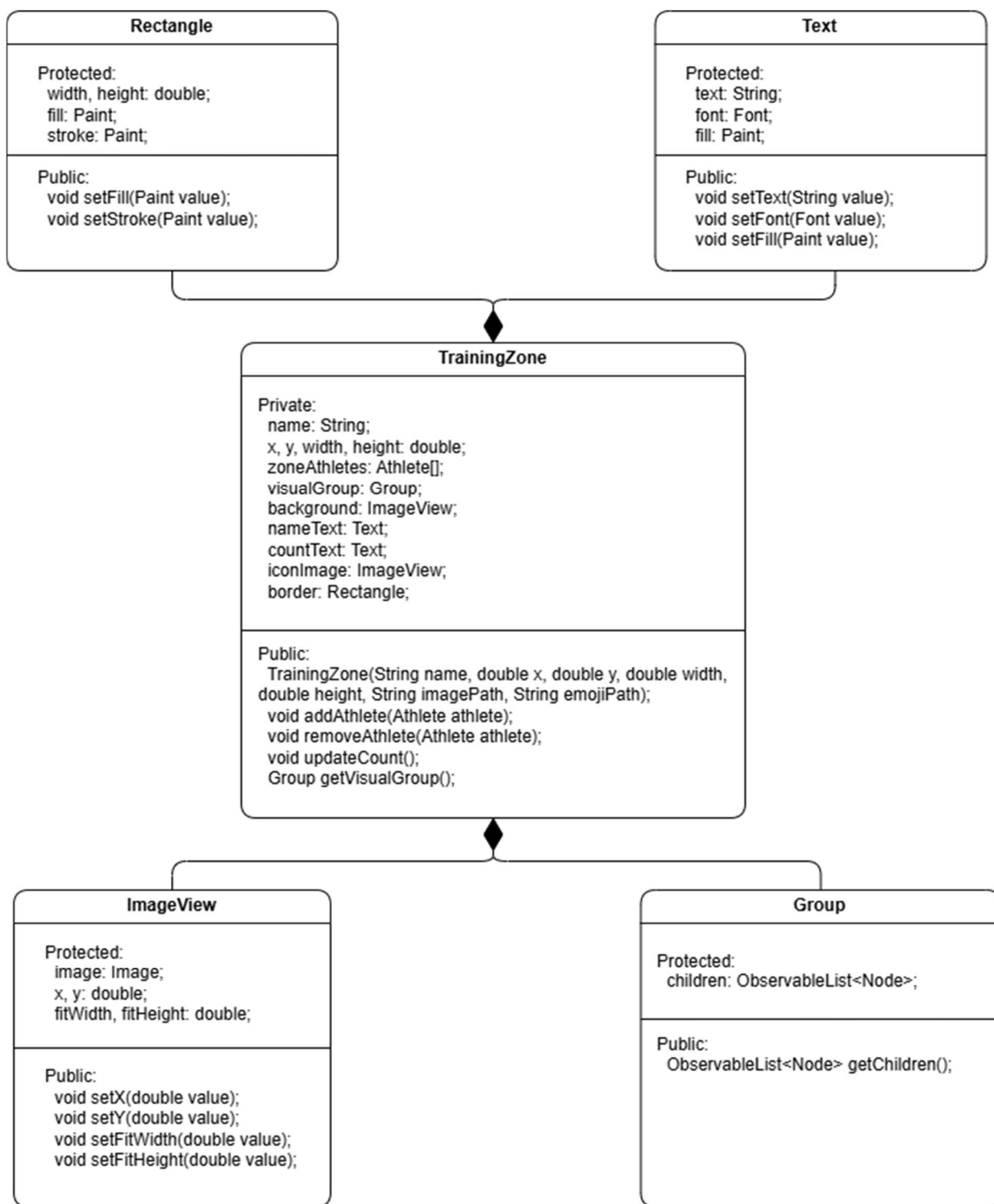


Рисунок 3.3 - Діаграма композиції

Ці графічні елементи ініціалізуються та створюються безпосередньо всередині класу **TrainingZone**, формуючи його візуальне відображення на екрані. У випадку видалення об'єкта тренувальної зони, всі її складові графічні компоненти також будуть автоматично знищені збирачем сміття, оскільки вони не мають логічного сенсу для існування окремо.

3.4 Діаграма асоціації

Асоціація — це зв'язок між двома класами, який дозволяє одному об'єкту взаємодіяти з іншим для обміну повідомленнями або даними. На рисунку 3.4 зображено діаграму асоціації між мікрооб'єктом *Athlete* та об'єктом тренувальної зони *TrainingZone*.



Рисунок 3.4 - Діаграма асоціації

Об'єкт атлета має поле `currentZone`, яке зберігає посилання на зону, в якій він зараз перебуває (або значення `null`, якщо він знаходиться поза зонами). У свою чергу, об'єкт *TrainingZone* містить масив `zoneAthletes`, який фіксує всіх атлетів, що наразі тренуються саме в ній. Завдяки цій двосторонній асоціації, при переміщенні атлета по сцені, система перевіряє перетин координат (метод `intersects()`) і викликає відповідні методи `addAthlete()` або `removeAthlete()` для взаємодії. Це забезпечує точне відображення кількості відвідувачів у конкретній зоні на екрані та оновлення статусу самого атлета.

3.5 Діаграма кооперації

Діаграма кооперації (англ. *communication diagram*) — це тип діаграми взаємодії, яка показує зв'язки між об'єктами, фокусуючись на їхній структурній організації та повідомленнях, якими вони обмінюються. На відміну від діаграми послідовності, тут акцент робиться на архітектурних взаємозв'язках об'єктів.



Рисунок 3.5 - Діаграма кооперації

У симуляторі «Tornado GYM» діаграма кооперації (див. рисунок 3.5) відображає взаємодію між головним класом програми `GymWorld` (що містить ігровий цикл `AnimationTimer`), мікрооб'єктом `Athlete` та об'єктом `TrainingZone`. Процес кооперації побудований на передачі сигналів під час переміщення персонажа. Об'єкт `GymWorld` виступає ініціатором перевірки координат. Він надсилає запит до `Athlete` для отримання його поточних меж у просторі за допомогою графічного вузла. Після виявлення перетину меж із зоною тренування, `GymWorld` передає об'єкт атлета як параметр у метод `addAthlete(athlete)` класу `TrainingZone`. Об'єкт тренувальної зони, отримавши повідомлення, модифікує свій внутрішній масив `zoneAthletes` і викликає внутрішній метод `updateCount()` для зміни текстового індикатора на екрані.

3.6 Діаграма послідовності

Діаграма послідовності (англ. *sequence diagram*) відображає взаємодію об'єктів у системі, впорядковану за часом. Вона демонструє часові лінії (*lifelines*) об'єктів та послідовність викликів методів у хронологічному порядку.

На рисунку 3.6 представлено діаграму послідовності процесу реєстрації та зміни статусу атлета при вході в тренувальну зону. Початковою подією є спрацювання ігрового таймера `AnimationTimer` у класі `GymWorld`. Послідовність дій виглядає так:

1. `GymWorld` викликає метод `getVisualGroup().getBoundsInParent()` у кожного об'єкта `Athlete` та `TrainingZone` для перевірки колізії.

2. При поверненні логічного значення `true` від методу `intersects()`, `GymWorld` надсилає повідомлення `removeAthlete(this)` до попередньої зони атлета (якщо вона була задана).
3. `GymWorld` викликає метод `addAthlete(a)` у нового об'єкта `TrainingZone`.
4. Клас `TrainingZone` всередині себе виконує метод `updateCount()`, який змінює колір та значення лічильника `countText`.
5. `GymWorld` викликає метод `setZoneStatus(true, emoji)` та `setCurrentZone(detectedZone)` у об'єкта `Athlete`, що призводить до відображення відповідної графічної іконки над персонажем та оновлення його внутрішнього стану.

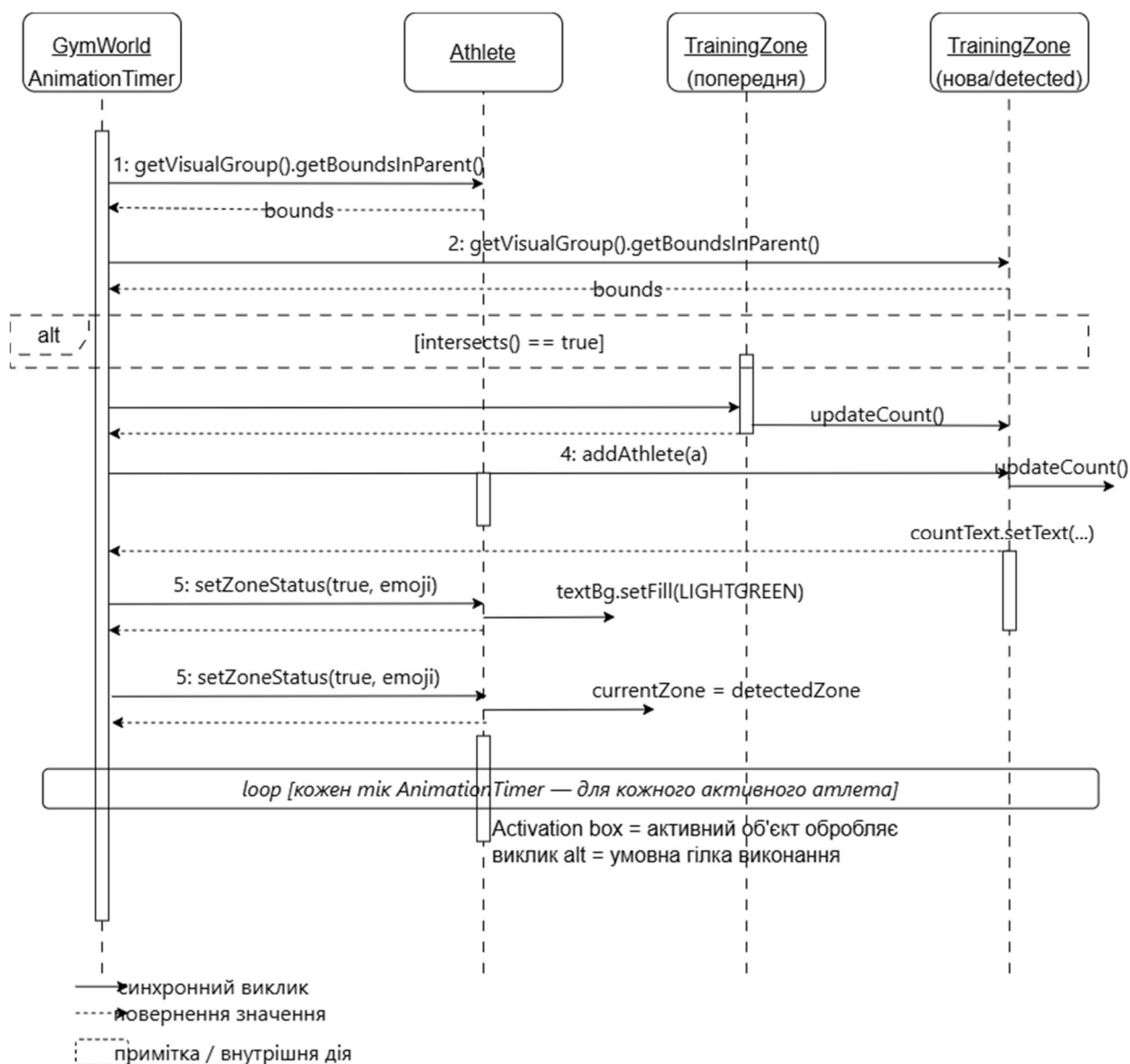


Рисунок 3.6 - Діаграма послідовності

Таким чином, описана послідовність взаємодій забезпечує безперервну синхронізацію просторового розташування атлетів із відповідними тренувальними зонами та своєчасне оновлення графічного інтерфейсу системи в реальному часі.

3.7 Діаграма стану

Діаграма станів (англ. state machine diagram) описує динамічну поведінку одного об'єкта у вигляді скінченного автомата. Вона фіксує всі можливі стани, у яких може перебувати об'єкт протягом свого життєвого циклу, а також події та умови, що викликають перехід з одного стану в інший.

Для мікрооб'єкта Athlete розроблено діаграму станів (рисунок 3.7), яка базується на зміні його координат та рівня енергії energyLvl. Виділено такі основні стани об'єкта:

- **Idle (Очікування / Спокій):** Початковий стан атлета при створенні на карті, коли параметри isActive = false, а координати не змінюються.
- **Moving (Переміщення):** Перехід у цей стан відбувається за події натискання клавіш керування (генерація подій у activeKeys). Текстура персонажа переміщується по сцені за допомогою методу move().
- **Training (Тренування):** Стан активується за умови перебування об'єкта всередині меж TrainingZone. У цьому стані запускається таймер зменшення energyLvl (витривалості) та оновлюється графічний елемент energyBar.
- **Exhausted / Resting (Втома / Відпочинок):** Автоматичний перехід сюди відбувається, коли energyLvl падає нижче критичної позначки. Об'єкт блокує можливість тренувань і виконує методи rest() або eat() (відновлення енергії), після чого повертається до стану спокою.

Діаграма стану: Athlete

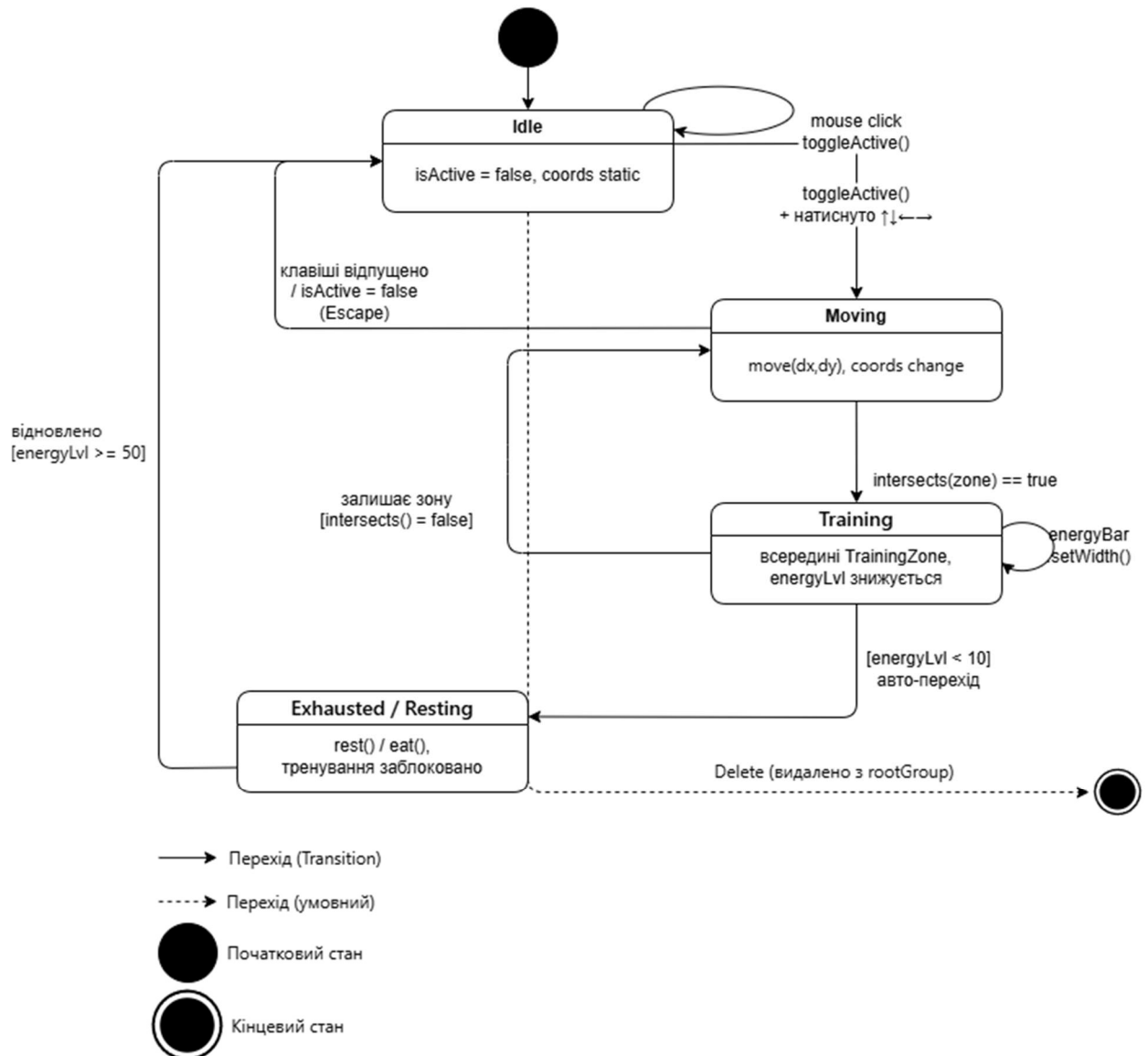


Рисунок 3.7 - Діаграма станів

Таким чином, розроблена діаграма повноцінно описує життєвий цикл об'єкта Athlete, деталізуючи його реакції на дії користувача, просторову взаємодію з ігровим середовищем та управління внутрішніми ресурсами.

4 КЕРІВНИЦТВО КОРИСТУВАЧА

Програма GymWorld реалізована у вигляді виконуваного JAR-архіву та не потребує традиційної процедури встановлення. Для початку роботи достатньо скопіювати файл GymWorld.jar до будь-якої зручної директорії на комп'ютері та запустити його подвійним клацанням лівої кнопки миші. Операційна система автоматично передасть керування встановленій JVM, яка виконає архів.

Необхідною умовою запуску є наявність встановленої Java Runtime Environment (JRE) версії 17 або вище. Якщо на комп'ютері встановлена лише JDK — запуск також можливий. Перевірити наявну версію Java можна командою `java -version` у консолі (термінал / командний рядок). Без Java програма не запуститься навіть у вигляді скомпільованого архіву.

Альтернативний спосіб запуску — безпосередньо через консоль: перейдіть до каталогу, який містить файл, та виконайте команду `java -jar GymWorld.jar`. Якщо ж у вас є скомпільовані .class-файли, переконайтесь, що шлях до бінарних файлів Java додано до змінної середовища PATH, після чого виконайте команду `java gymworld.GymWorld` зі кореневої директорії проєкту.

4.1 Головне вікно програми

Після успішного запуску відкривається головне вікно розміром 1920×1080 пікселів, розгорнуте на весь екран. Вікно представляє двовимірну ігрову карту спортивного залу GymWorld зі спортсменами, які рухаються у його просторі. У лівому верхньому куті розташована міні-карта, що відображає поточне положення всіх об'єктів у масштабі.

Основну частину екрану займає ігрове поле з трьома функціональними зонами: зоною реєстрації (Front Desk), кардіо-зоною (Cardio Area) та зоною з вагами (Weights Area). Кожна зона має власне фонове зображення та відображає кількість атлетів, що знаходяться в її межах. Поточна кількість відображається в підписі зони: червоним кольором — якщо зона порожня, чорним — якщо є присутні атлети.

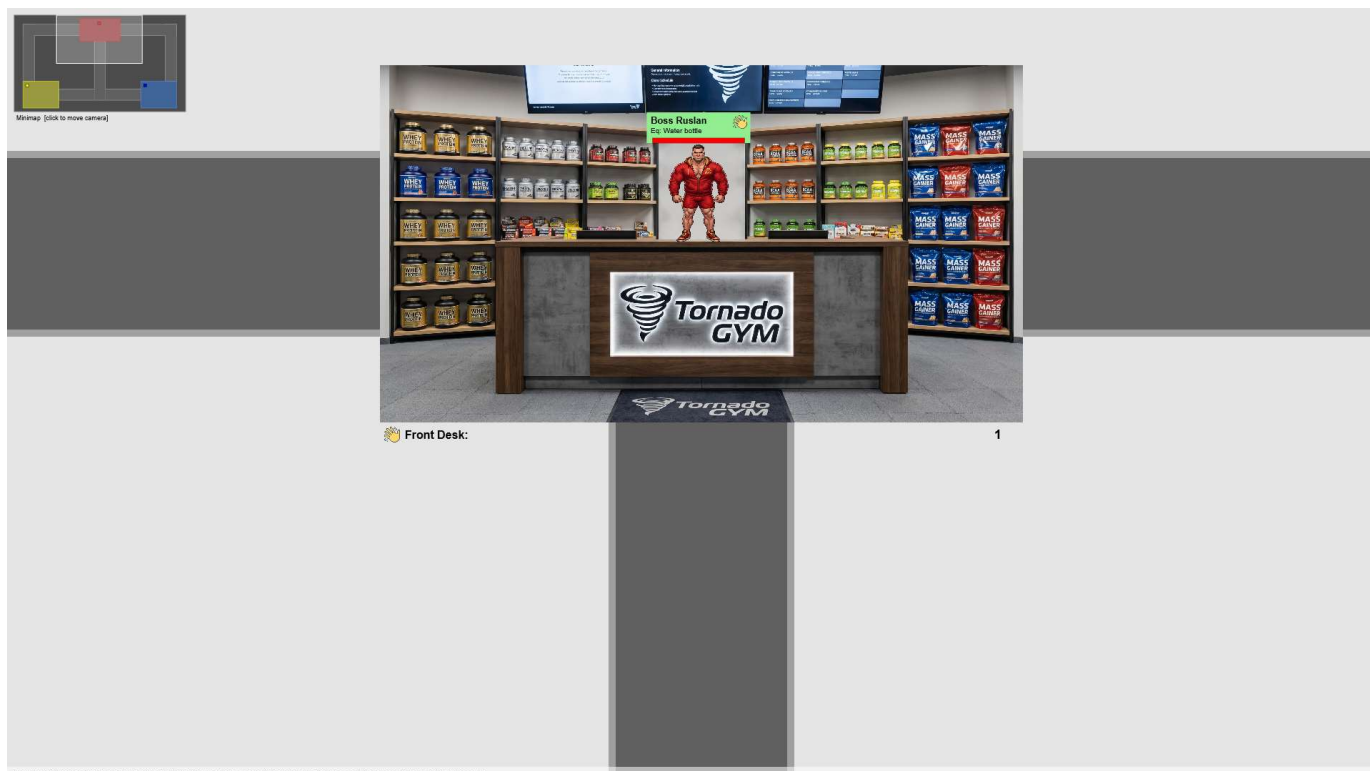


Рисунок 4.1 – Вигляд програми при коректному запуску

У лівій нижній частині вікна(див. рисунок 4.1) знаходиться рядок гарячих клавіш, що відображає доступні команди керування програмою: переміщення камери клавішами Arrows/WASD, додавання нового атлета (INSERT), пошук (F), запити (G), автоматичний рух (P), виділення всіх об'єктів (B), клонування (Ctrl+C), видалення активного атлета (DEL), збереження стану гри (Ctrl+S), завантаження (Ctrl+L) та скасування виділення (ESC). Цей рядок слугує постійною візуальною підказкою для користувача і не потребує звернення до окремої довідки.

4.2 Додавання нового атлета

Для додавання нового атлета до залу передбачено два способи: натиснути гарячу клавішу INSERT на клавіатурі. Відкриється модальне діалогове вікно створення нового персонажа.

У вікні (рисунок 4.2) необхідно заповнити такі поля:

1. Тип (Type) - визначає клас атлета: звичайний Athlete, PowerLifter або GymBoss. Від типу залежать зовнішній вигляд персонажа та його особливості поведінки.
2. Ім'я (Name) - текстовий рядок з ім'ям атлета, що відобразиться над його спрайтом.
3. Вага (Weight, кг) - числове значення ваги атлета. За замовчуванням встановлено 80 кг.
4. Обладнання (Equipment) - прапорець «Has Equipment» дозволяє вказати спорядження атлета. Якщо прапорець встановлено, зі спадного списку обирається конкретний тип інвентарю: гантелі, штанга, скакалка, тренажер тощо.
5. Зона появи (Spawn Zone) - місце, де атлет з'явиться на карті: зона рецепції, кардіо або зона з вагами.

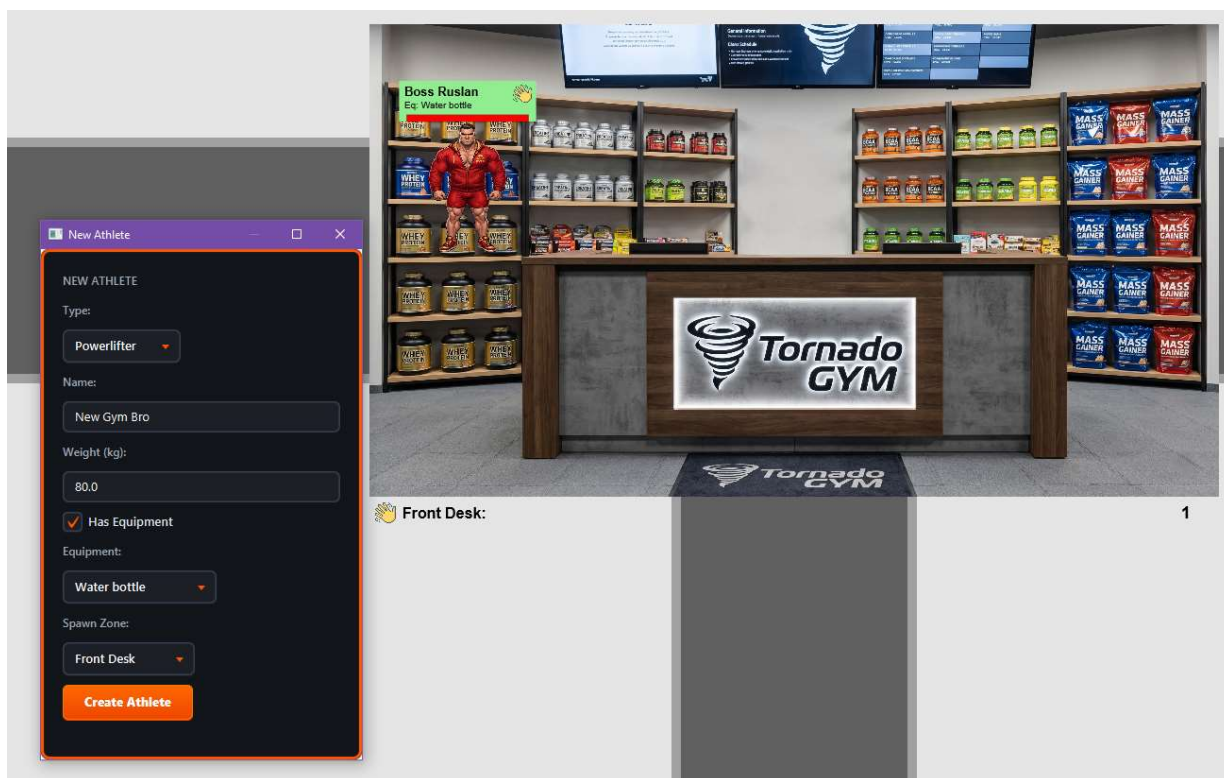


Рисунок 4.2 – Інтерфейс додавання нового атлета

Після заповнення всіх полів натисніть кнопку «Create Athlete». Новий атлет буде розміщений у вибраній зоні та відразу стане видимим на ігровому полі.

4.3 Переміщення атлетів

Щоб розпочати керування атлетом, потрібно спочатку зробити його активним — клацнути по ньому лівою кнопкою миші. Активний атлет виділяється анімованою райдужною рамкою навколо спрайта. Можна одночасно мати кілька активних персонажів — усі вони переміщатимуться одночасно.

Керування рухом здійснюється клавішами WASD або стрілками клавіатури:

- W / ↑ — рух вгору;
- S / ↓ — рух вниз;
- A / ← — рух ліворуч;
- D / → — рух праворуч.

Коли атлет переходить у межі однієї з тренувальних зон, його інформаційна панель змінює колір фону на світло-зелений, а поруч з іменем з'являється емоцій-іконка, що відповідає поточній зоні. Одночасно лічильник атлетів у підписі зони збільшується. При виході з зони — усі зміни скасовуються.

Крім ручного керування, атлети підтримують режим автоматичного руху. В цьому режимі персонаж самостійно рухається по заданій траєкторії, відбиваючись від меж ігрового поля. Увімкнути або вимкнути автоматичний рух можна через натиснення англійської клавіші P. Заборонити вихід з зони можна на англійську клавішу B.

4.4 Редагування атлета

Для редагування характеристик вже створеного атлета слід клацнути по ньому правою кнопкою миші. Відкриється модальне вікно редагування (рисунок 4.3), що містить ті самі поля, що й вікно створення, але поле «Тип» заблоковане для зміни — тип атлета визначається при його створенні та є незмінним.

У вікні редагування доступні такі поля: ім'я, вага, рівень енергії (Energy Level, від 1 до 100) та обладнання. Рівень енергії впливає на довжину кольорової

шкали у верхній частині спрайта атлета: повна шкала — 100 одиниць, порожня — 1. Зміни застосовуються негайно після натискання кнопки «Save Changes».

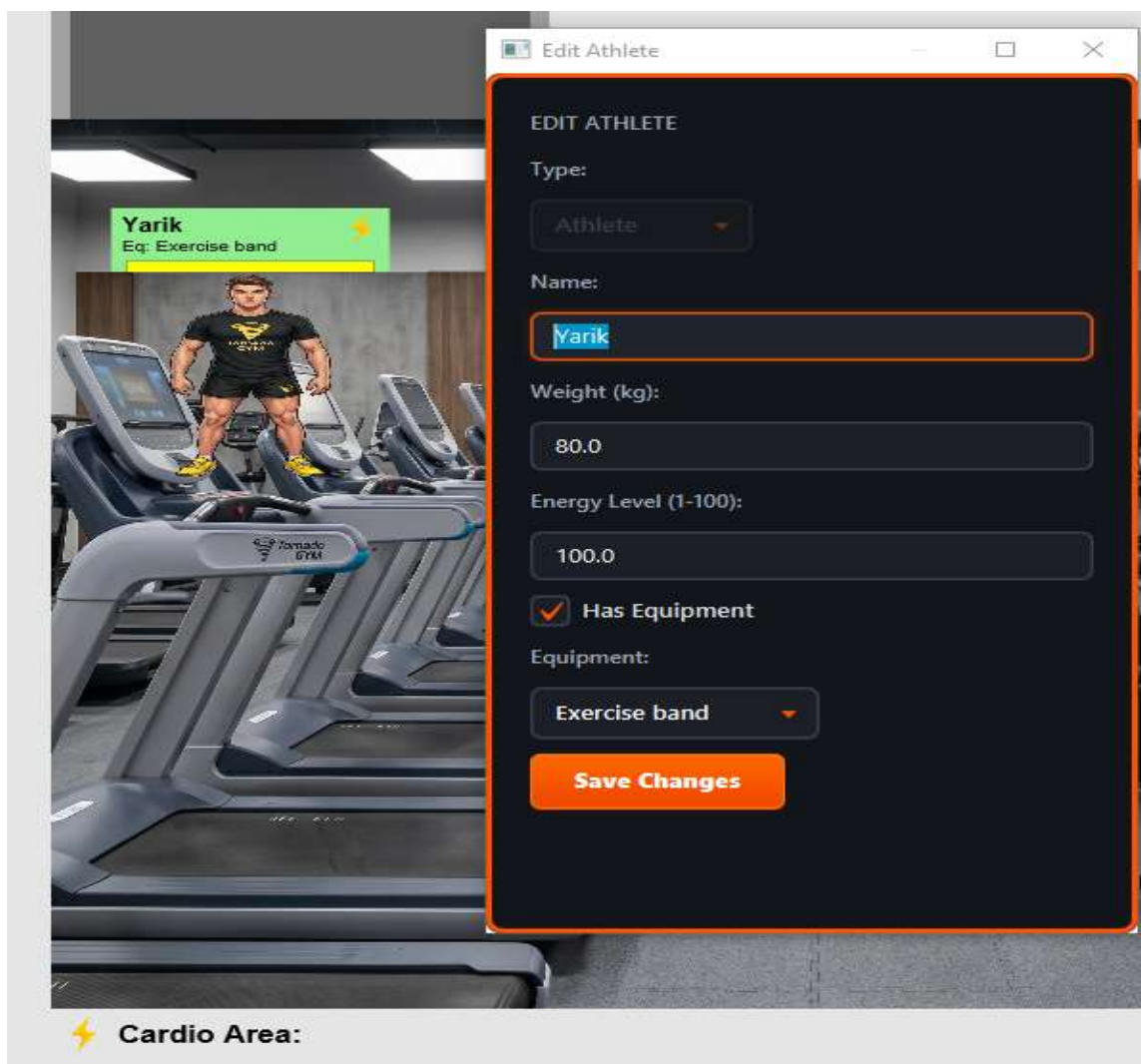


Рисунок 4.3 – Вікно редагування характеристик атлета

Отже, даний функціонал дозволяє динамічно оновлювати поточні параметри об'єкта в реальному часі, гарантуючи при цьому суворе збереження його базової типізації, яка фіксується ще на етапі створення.

4.5 Видалення та клонування атлетів

Щоб видалити активного атлета зі сцени, скористайтесь клавішею DELETE. Операція незворотна в межах поточного сеансу, якщо стан не був збережений попередньо.

Клонування дозволяє швидко створити копію існуючого атлета зі збереженням усіх його параметрів. Клон розміщується поруч із оригіналом зі зміщенням праворуч. Для клонування активного атлета скористайтесь комбінацією клавіш Ctrl+C. Клон є повністю незалежним об'єктом і не прив'язаний до поточної тренувальної зони оригіналу.

4.6 Сортування та статистика

Програма підтримує сортування відображуваного списку атлетів за трьома критеріями: за ім'ям (алфавітний порядок), за вагою та за рівнем енергії[14, 15]. Доступ до сортування здійснюється через натискання англійської клавіші G на клавіатурі. Порядок сортування можна змінювати між зростанням та спаданням.

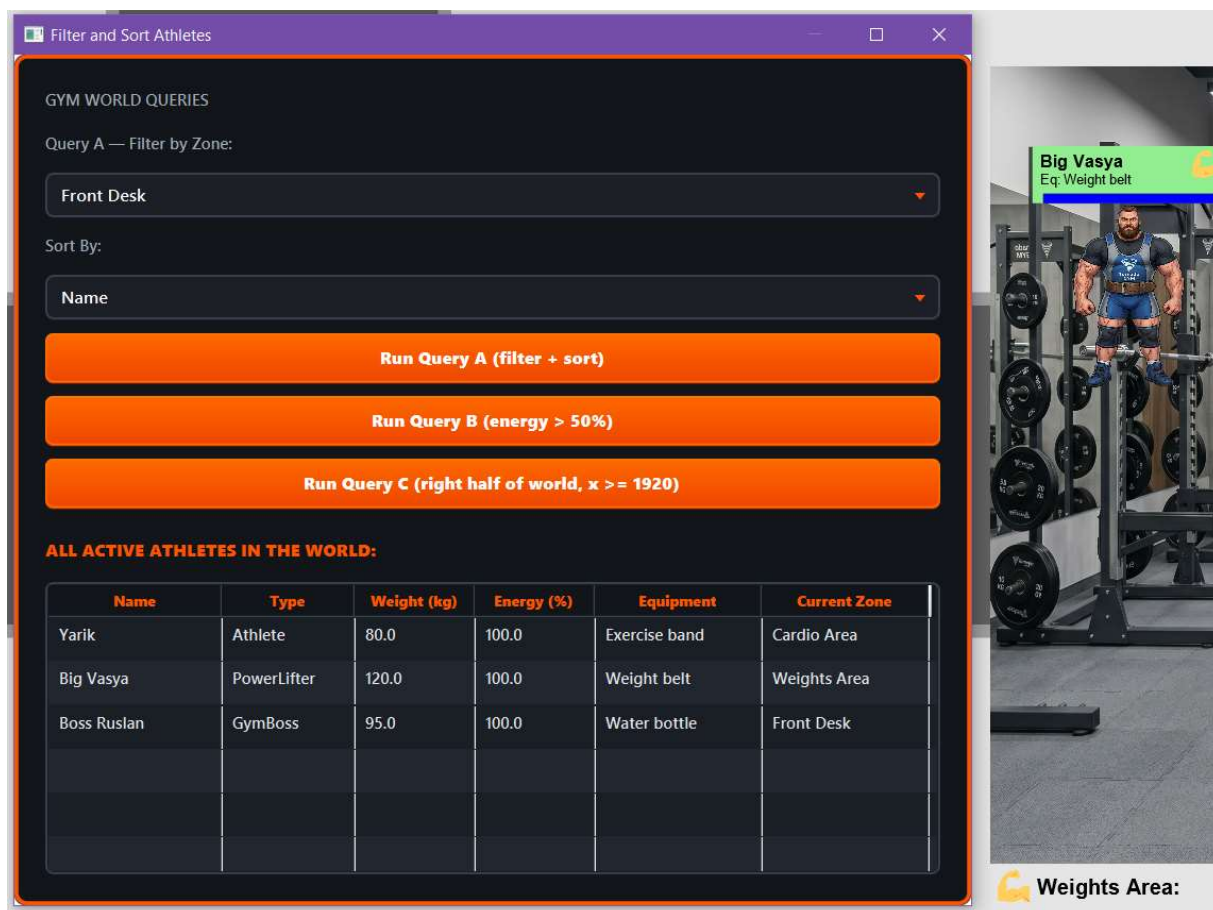


Рисунок 4.4 – Вікно статистики атлетів

Завдяки зручному табличному поданню(див. рисунок 4.4) та можливості використовувати спеціальні запити для фільтрації, користувач отримує потужний інструмент для швидкого аналізу та впорядкування інформації про всіх активних учасників симуляції.

4.7 Збереження та завантаження стану

Програма підтримує збереження поточного стану ігрового поля у бінарний файл формату .gum (серіалізація Java-об'єктів). Збереження здійснюється через натискання комбінації клавіш Ctrl+S. Для вибору місця збереження відкривається стандартний системний діалог.

Завантаження раніше збереженого стану виконується через комбінацію клавіш Ctrl+L. При завантаженні всі поточні об'єкти сцени замінюються збереженими. Важливо враховувати, що візуальні компоненти (JavaFX-вузли) не

серіалізуються — вони відтворюються автоматично після завантаження через метод `initVisuals()`.

4.8 Повідомлення про помилки та особливі ситуації

У разі введення некоректних числових значень (наприклад, нечислового тексту у поле ваги чи енергії) програма автоматично підставляє значення за замовчуванням: вага — 80 кг, енергія — без змін. Таким чином, жодних критичних помилок через некоректне введення не виникне.

Якщо запуск програми завершується помилкою, найбільш вірогідними причинами є: відсутність або невідповідна версія Java (нижче 17), пошкоджений файл архіву, або відсутність необхідних графічних ресурсів у структурі JAR. У такому разі рекомендується запустити програму через консоль — це дозволить побачити детальне повідомлення про виниклу помилку в стандартному потоці виведення.

ВИСНОВКИ

У результаті виконання курсової роботи було розроблено програмний застосунок «Tornado GYM» — інтерактивний симулятор спортивного залу з графічним інтерфейсом на платформі JavaFX. Реалізовано трирівневу ієрархію мікрооб'єктів-атлетів (Athlete → PowerLifter → GymBoss) та три макрооб'єкти-зони (Front Desk, Cardio Area, Weights Area), що функціонують у межах універсального об'єкта GymWorld. В ході проєктування і реалізації системи вдалося застосувати всі ключові принципи об'єктно-орієнтованого програмування: абстракцію, інкапсуляцію, успадкування та поліморфізм. Глибинне копіювання реалізовано через інтерфейс Cloneable, збереження стану симуляції — через серіалізацію Java-об'єктів[8, 9]. Додатково продемонстровано використання лямбда-виразів та Stream API для реалізації запитів і сортування атлетів за трьома критеріями[14, 15]. Проаналізовано актуальність теми: за даними ВООЗ (2022)[3], понад 1,4 мільярда дорослих у всьому світі ведуть недостатньо активний спосіб життя, що підтверджує соціальну значущість освітньо-мотиваційних інструментів на кшталт розробленого симулятора.

Під час роботи над проєктом набуто практичні навички програмування у середовищі IntelliJ IDEA, поглиблено знання мови Java та бібліотеки JavaFX для побудови графічного інтерфейсу, анімацій та обробки подій. Отримано досвід проєктування архітектури застосунку з використанням UML-діаграм (наслідування, агрегації, композиції, асоціації, кооперації, послідовності та станів). Розроблений продукт відповідає усім вимогам технічного завдання: підтримує мінікарту з навігацією, автоматичний рух об'єктів, діалогові вікна для керування атлетами, збереження та завантаження стану через FileChooser, а також реалізує не менше трьох запитів для підрахунку мікрооб'єктів і встановлення критеріїв сортування. Набутий досвід є цінним фундаментом для подальшої розробки складних десктопних застосунків із насиченим графічним інтерфейсом на платформі Java.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Cózar-Gutiérrez R. et al. Positive and Negative Impacts of Gamification on the Fitness Industry. *European Journal of Investigation in Health, Psychology and Education*. 2023. Vol. 13, no. 8. P. 1342–1356. URL: <https://doi.org/10.3390/ejihpe13080103> (дата звернення: 01.06.2026).
2. Mazeas A. et al. Evaluating the Effectiveness of Gamification on Physical Activity: Systematic Review and Meta-analysis of Randomized Controlled Trials. *Journal of Medical Internet Research*. 2022. Vol. 24, no. 1. P. e26779. URL: <https://doi.org/10.2196/26779> (дата звернення: 01.06.2026).
3. World Health Organization. *Global Status Report on Physical Activity 2022*. Geneva : WHO, 2022. 92 p. URL: <https://www.who.int/publications/i/item/9789240059153> (дата звернення: 01.06.2026).
4. Sharan K. *Learn JavaFX 17: Building User Experience and Interfaces with Java*. 2nd ed. New York : Apress, 2022. 1100 p.
5. Deitel P., Deitel H. *Java How to Program*. 11th ed. Hoboken : Pearson, 2018. 1006 p.
6. Oracle Corporation. *JavaFX Overview (Release 8)*. Oracle Documentation. 2024. URL: <https://docs.oracle.com/javase/8/javafx/get-started-tutorial/jfx-overview.htm> (дата звернення: 01.06.2026).
7. OpenJFX. *JavaFX Documentation*. OpenJFX Community. 2024. URL: <https://openjfx.io/javadoc/17/> (дата звернення: 01.06.2026).
8. Oracle Corporation. *Java Object Serialization*. Oracle Documentation. 2024. URL: <https://docs.oracle.com/javase/8/docs/technotes/guides/serialization/index.html> (дата звернення: 01.06.2026).
9. Oracle Corporation. *Serializable (Java SE 17 & JDK 17)*. Oracle API Documentation. 2024. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/Serializable.html> (дата звернення: 01.06.2026).
10. Goncalves A. *Beginning Java EE 7*. New York : Apress, 2013. 608 p.

11. Fowler M. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3rd ed. Boston : Addison-Wesley, 2003. 208 p.
12. Bloch J. Effective Java. 3rd ed. Boston : Addison-Wesley, 2018. 412 p.
13. Gamma E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Boston : Addison-Wesley, 1994. 395 p.
14. Oracle Corporation. The Java Tutorials — Lambda Expressions. Oracle Documentation. 2024. URL: <https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html> (дата звернення: 01.06.2026).
15. Oracle Corporation. Stream (Java SE 17 & JDK 17). Oracle API Documentation. 2024. URL: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/Stream.html> (дата звернення: 01.06.2026).

ДОДАТОК А

Лістинг програми (обов'язковий) Launcher.java

```
package gymworld;
import javafx.application.Application;

public class Launcher {
    public static void main(String[] args) {
        Application.launch(GymWorld.class, args);
    }
}
```

GymWorld.java

```
package gymworld;

import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyCombination;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.scene.text.Text;
import javafx.stage.Screen;
import javafx.stage.Stage;
import javafx.stage.StageStyle;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class GymWorld extends Application {

    public static double WORLD_W = 3840;
    public static double WORLD_H = 2160;

    public static double SCREEN_W = 960;
    public static double SCREEN_H = 540;

    public static Stage mainStage;
    public static Group rootGroup;
    public static List<Athlete> athletes = new ArrayList<>();
    public static List<TrainingZone> zones = new ArrayList<>();
    public static final List<Rectangle> corridors = new ArrayList<>();
```

```

private final Set<KeyCode> activeKeys = new HashSet<>();
public static boolean solidMode = false;

public static double camX = 0;
public static double camY = 0;
private static final double CAM_SPEED = 6;

private static double FD_W = 900;
private static double FD_H = 500;
private static double FD_X;
private static double FD_Y;

private static double CA_W = 800;
private static double CA_H = 600;
private static double CA_X;
private static double CA_Y;

private static double WA_W = 800;
private static double WA_H = 600;
private static double WA_X;
private static double WA_Y;

private static double COR_W = 260;

public static Minimap minimap;
private static Rectangle worldClip;
private Rectangle hudBg;
private Text hudText;

@Override
public void start(Stage stage) {
    mainStage = stage;

    javafx.geometry.Rectangle2D screenBounds = Screen.getPrimary().getBounds();
    SCREEN_W = screenBounds.getWidth();
    SCREEN_H = screenBounds.getHeight();
    WORLD_W = SCREEN_W * 2.0;
    WORLD_H = SCREEN_H * 2.0;

    FD_X = (WORLD_W - FD_W) / 2.0;
    FD_Y = 80;

    CA_X = 200;
    CA_Y = WORLD_H - CA_H - 80;

    WA_X = WORLD_W - WA_W - 200;
    WA_Y = WORLD_H - WA_H - 80;

    rootGroup = new Group();
    Group sceneRoot = new Group(rootGroup);

    Scene scene = new Scene(sceneRoot, SCREEN_W, SCREEN_H);

```

```

scene.setFill(Color.web("#E5E5E5"));

worldClip = new Rectangle(0, 0, SCREEN_W, SCREEN_H);
rootGroup.setClip(worldClip);

TrainingZone frontDesk = new TrainingZone("Front Desk", FD_X, FD_Y, FD_W, FD_H,
    "/gymworld/front_desk.png", "/gymworld/wave.png");
TrainingZone cardioArea = new TrainingZone("Cardio Area", CA_X, CA_Y, CA_W, CA_H,
    "/gymworld/cardio_area.png", "/gymworld/lightning.png");
TrainingZone weightsArea = new TrainingZone("Weights Area", WA_X, WA_Y, WA_W, WA_H,
    "/gymworld/weights_area.png", "/gymworld/bicep.png");

zones.add(frontDesk);
zones.add(cardioArea);
zones.add(weightsArea);

Rectangle lcHBase = new Rectangle(CA_X, FD_Y + FD_H / 2.0 - COR_W / 2.0, FD_X - CA_X, COR_W);
Rectangle lcHInner = new Rectangle(CA_X + 10, FD_Y + FD_H / 2.0 - COR_W / 2.0 + 10, FD_X - CA_X, COR_W - 20);
Rectangle lcVBase = new Rectangle(CA_X, FD_Y + FD_H / 2.0 - COR_W / 2.0, COR_W, CA_Y - (FD_Y + FD_H / 2.0 - COR_W / 2.0));
Rectangle lcVInner = new Rectangle(CA_X + 10, FD_Y + FD_H / 2.0 - COR_W / 2.0 + 10, COR_W - 20, CA_Y - (FD_Y + FD_H / 2.0 - COR_W /
2.0) - 10);

Rectangle rcHBase = new Rectangle(FD_X + FD_W, FD_Y + FD_H / 2.0 - COR_W / 2.0, (WA_X + WA_W) - (FD_X + FD_W), COR_W);
Rectangle rcHInner = new Rectangle(FD_X + FD_W, FD_Y + FD_H / 2.0 - COR_W / 2.0 + 10, (WA_X + WA_W - 10) - (FD_X + FD_W),
COR_W - 20);
Rectangle rcVBase = new Rectangle(WA_X + WA_W - COR_W, FD_Y + FD_H / 2.0 - COR_W / 2.0, COR_W, WA_Y - (FD_Y + FD_H / 2.0 -
COR_W / 2.0));
Rectangle rcVInner = new Rectangle(WA_X + WA_W - COR_W + 10, FD_Y + FD_H / 2.0 - COR_W / 2.0 + 10, COR_W - 20, WA_Y - (FD_Y +
FD_H / 2.0 - COR_W / 2.0) - 10);

Rectangle ccVBase = new Rectangle(FD_X + FD_W / 2.0 - COR_W / 2.0, FD_Y + FD_H, COR_W, (CA_Y + CA_H / 2.0 + COR_W / 2.0) -
(FD_Y + FD_H));
Rectangle ccVInner = new Rectangle(FD_X + FD_W / 2.0 - COR_W / 2.0 + 10, FD_Y + FD_H, COR_W - 20, (CA_Y + CA_H / 2.0 + COR_W /
2.0 - 10) - (FD_Y + FD_H));
Rectangle ccLBase = new Rectangle(CA_X + CA_W, CA_Y + CA_H / 2.0 - COR_W / 2.0, (FD_X + FD_W / 2.0 - COR_W / 2.0) - (CA_X +
CA_W), COR_W);
Rectangle ccLInner = new Rectangle(CA_X + CA_W, CA_Y + CA_H / 2.0 - COR_W / 2.0 + 10, (FD_X + FD_W / 2.0 - COR_W / 2.0 + 10) -
(CA_X + CA_W), COR_W - 20);
Rectangle ccRBase = new Rectangle(FD_X + FD_W / 2.0 + COR_W / 2.0, WA_Y + WA_H / 2.0 - COR_W / 2.0, WA_X - (FD_X + FD_W / 2.0 +
COR_W / 2.0), COR_W);
Rectangle ccRInner = new Rectangle(FD_X + FD_W / 2.0 + COR_W / 2.0 - 10, WA_Y + WA_H / 2.0 - COR_W / 2.0 + 10, WA_X - (FD_X +
FD_W / 2.0 + COR_W / 2.0 - 10), COR_W - 20);

lcHBase.setFill(Color.web("#A0A0A0"));
lcHInner.setFill(Color.web("#606060"));
lcVBase.setFill(Color.web("#A0A0A0"));
lcVInner.setFill(Color.web("#606060"));

rcHBase.setFill(Color.web("#A0A0A0"));
rcHInner.setFill(Color.web("#606060"));
rcVBase.setFill(Color.web("#A0A0A0"));
rcVInner.setFill(Color.web("#606060"));

```

```

ccVBase.setFill(Color.web("#A0A0A0"));
ccVInner.setFill(Color.web("#606060"));
ccLBase.setFill(Color.web("#A0A0A0"));
ccLInner.setFill(Color.web("#606060"));
ccRBase.setFill(Color.web("#A0A0A0"));
ccRInner.setFill(Color.web("#606060"));

corridors.clear();
corridors.addAll(List.of(
    lcHBase, lcVBase,
    rcHBase, rcVBase,
    ccVBase, ccLBase, ccRBase,
    lcHInner, lcVInner,
    rcHInner, rcVInner,
    ccVInner, ccLInner, ccRInner
));

rootGroup.getChildren().addAll(
    lcHBase, lcVBase,
    rcHBase, rcVBase,
    ccVBase, ccLBase, ccRBase,
    lcHInner, lcVInner,
    rcHInner, rcVInner,
    ccVInner, ccLInner, ccRInner,
    frontDesk.getVisualGroup(),
    cardioArea.getVisualGroup(),
    weightsArea.getVisualGroup()
);

Athlete testAthlete = new Athlete("Yarik", 20, 80.0, 100.0,
    new Equipment("Exercise band", 5, "kg"), CA_X + 60, CA_Y + 60);
Athlete powerLifter = new PowerLifter("Big Vasya", 35, 120.0, 100.0,
    new Equipment("Weight belt", 10, "kg"), WA_X + 60, WA_Y + 60);
Athlete gymBoss = new GymBoss("Boss Ruslan", 45, 95.0, 100.0,
    new Equipment("Water bottle", 1, "kg"), FD_X + 60, FD_Y + 60);

athletes.add(testAthlete);
athletes.add(powerLifter);
athletes.add(gymBoss);
for (Athlete a : athletes) assignToZoneInitially(a);

rootGroup.getChildren().addAll(
    testAthlete.getVisualGroup(),
    powerLifter.getVisualGroup(),
    gymBoss.getVisualGroup()
);

minimap = new Minimap(SCREEN_W, SCREEN_H);
sceneRoot.getChildren().add(minimap.getRoot());

double hudH = 20;
hudBg = new Rectangle(0, SCREEN_H - hudH, SCREEN_W, hudH);
hudBg.setFill(Color.web("#12161A", 0.90));

```

```

hudBg.setStroke(Color.web("#FF5500", 0.8));
hudBg.setStrokeWidth(1);
hudBg.setMouseTransparent(true);
hudText = new Text(8, SCREEN_H - 5,
    "Arrows/WASD = camera/move | INSERT = add | F = find | G = queries | P = auto | " +
    "B = solid | Ctrl+C = clone | DEL = delete | Ctrl+S = save | Ctrl+L = load | ESC = deselect");
hudText.setFont(Font.font("Segoe UI", javafx.scene.text.FontWeight.BOLD, 10));
hudText.setFill(Color.web("#E0E4E8"));
hudText.setMouseTransparent(true);
sceneRoot.getChildren().addAll(hudBg, hudText);

scene.setOnKeyPressed(event -> {
    activeKeys.add(event.getCode());

    if (event.getCode() == KeyCode.INSERT) {
        activeKeys.remove(KeyCode.INSERT);
        Athlete newAthlete = InsertDialog.display();
        if (newAthlete != null) {
            athletes.add(newAthlete);
            assignToZoneInitially(newAthlete);
            rootGroup.getChildren().add(newAthlete.getVisualGroup());
            newAthlete.getVisualGroup().toFront();
        }
    }

    if (event.getCode() == KeyCode.F) {
        activeKeys.remove(KeyCode.F);
        SearchDialog.display();
    }

    if (event.getCode() == KeyCode.G) {
        activeKeys.remove(KeyCode.G);
        FilterDialog.display();
    }

    if (event.getCode() == KeyCode.P) {
        activeKeys.remove(KeyCode.P);
        athletes.stream().filter(a -> !a.isActive()).forEach(a -> {
            a.isAutoMoving = !a.isAutoMoving;
            if (a.isAutoMoving) {
                a.autoDx = (Math.random() > 0.5 ? 2.5 : -2.5);
                a.autoDy = (Math.random() > 0.5 ? 2.5 : -2.5);
            }
        });
    }

    if (event.getCode() == KeyCode.B) {
        activeKeys.remove(KeyCode.B);
        solidMode = !solidMode;
    }

    if (event.isControlDown() && event.getCode() == KeyCode.C) {
        List<Athlete> clones = new ArrayList<>();
        athletes.stream().filter(Athlete::isActive).forEach(a -> {
            try {
                Athlete cl = (Athlete) a.clone();
                clones.add(cl);
                rootGroup.getChildren().add(cl.getVisualGroup());
            }
        });
    }
});

```

```

        cl.getVisualGroup().toFront();
    } catch (CloneNotSupportedException ex) { ex.printStackTrace(); }
});
athletes.addAll(clones);
}
if (event.isControlDown() && event.getCode() == KeyCode.S) {
    activeKeys.remove(KeyCode.S);
    SerializationManager.save();
}
if (event.isControlDown() && event.getCode() == KeyCode.L) {
    activeKeys.remove(KeyCode.L);
    SerializationManager.load();
}
if (event.getCode() == KeyCode.ESCAPE) {
    athletes.stream().filter(Athlete::isActive).forEach(Athlete::toggleActive);
}
if (event.getCode() == KeyCode.DELETE) {
    List<Athlete> toRemove = athletes.stream().filter(Athlete::isActive)
        .collect(java.util.stream.Collectors.toList());
    toRemove.forEach(a -> {
        if (a.getCurrentZone() != null) a.getCurrentZone().removeAthlete(a);
        a.removeFromScene();
    });
    athletes.removeAll(toRemove);
}
});

scene.setOnKeyReleased(event -> activeKeys.remove(event.getCode()));

AnimationTimer timer = new AnimationTimer() {
    @Override
    public void handle(long now) {
        double dcx = 0, dcy = 0;
        if (activeKeys.contains(KeyCode.LEFT) || activeKeys.contains(KeyCode.A)) dcx = -CAM_SPEED;
        if (activeKeys.contains(KeyCode.RIGHT) || activeKeys.contains(KeyCode.D)) dcx = CAM_SPEED;
        if (activeKeys.contains(KeyCode.UP) || activeKeys.contains(KeyCode.W)) dcy = -CAM_SPEED;
        if (activeKeys.contains(KeyCode.DOWN) || activeKeys.contains(KeyCode.S)) dcy = CAM_SPEED;

        boolean anyActive = athletes.stream().anyMatch(Athlete::isActive);

        if (!anyActive && (dcx != 0 || dcy != 0)) {
            camX = Math.max(0, Math.min(WORLD_W - SCREEN_W, camX + dcx));
            camY = Math.max(0, Math.min(WORLD_H - SCREEN_H, camY + dcy));
            applyCamera();
        }

        for (Athlete a : athletes) {
            if (a.isActive() && (dcx != 0 || dcy != 0)) {
                double nextX = a.getX() + dcx;
                double nextY = a.getY() + dcy;
                if (canMoveTo(a, nextX, nextY)) {
                    a.move(dcx, dcy);
                    handleCollisions(a);
                }
            }
        }
    }
};

```



```

        updateZone(a);
        adjustCameraToAthlete(a);
    }
} else if (!a.isActive() && a.isAutoMoving) {
    autoMove(a);
}
}

minimap.update(rootGroup.getTranslateX(), rootGroup.getTranslateY());
}
};
timer.start();

stage.initStyle(StageStyle.UNDECORATED);
stage.setTitle("GymWorld");
stage.setScene(scene);
stage.setFullScreen(true);
stage.setFullScreenExitKeyCombination(KeyCombination.NO_MATCH);
stage.show();

scene.widthProperty().addListener((obs, oldW, newW) -> onScreenResize(newW.doubleValue(), scene.getHeight()));
scene.heightProperty().addListener((obs, oldH, newH) -> onScreenResize(scene.getWidth(), newH.doubleValue()));
}

private void onScreenResize(double w, double h) {
    SCREEN_W = w;
    SCREEN_H = h;
    worldClip.setWidth(w);
    worldClip.setHeight(h);
    if (hudBg != null) { hudBg.setY(h - 20); hudBg.setWidth(w); }
    if (hudText != null) { hudText.setY(h - 5); }
    if (minimap != null) minimap.resize(w, h);
    camX = Math.min(camX, WORLD_W - w);
    camY = Math.min(camY, WORLD_H - h);
    applyCamera();
}

public static void applyCamera() {
    rootGroup.setTranslateX(-camX);
    rootGroup.setTranslateY(-camY);
    worldClip.setX(camX);
    worldClip.setY(camY);
}

private void adjustCameraToAthlete(Athlete a) {
    double margin = 120.0;
    double screenX = a.getX() - camX;
    double screenY = a.getY() - camY;
    double targetCamX = camX;
    double targetCamY = camY;

    if (screenX < margin) {
        targetCamX = a.getX() - margin;
    }
}

```

```

    } else if (screenX > SCREEN_W - Athlete.IMG_W - margin) {
        targetCamX = a.getX() - (SCREEN_W - Athlete.IMG_W - margin);
    }

    if (screenY < margin) {
        targetCamY = a.getY() - margin;
    } else if (screenY > SCREEN_H - Athlete.TOTAL_H - margin) {
        targetCamY = a.getY() - (SCREEN_H - Athlete.TOTAL_H - margin);
    }

    targetCamX = Math.max(0, Math.min(WORLD_W - SCREEN_W, targetCamX));
    targetCamY = Math.max(0, Math.min(WORLD_H - SCREEN_H, targetCamY));

    if (targetCamX != camX || targetCamY != camY) {
        camX = targetCamX;
        camY = targetCamY;
        applyCamera();
    }
}

private void autoMove(Athlete a) {
    if (a.autoDx == 0 && a.autoDy == 0) {
        a.autoDx = (Math.random() > 0.5 ? 2.5 : -2.5);
        a.autoDy = (Math.random() > 0.5 ? 2.5 : -2.5);
    }

    double nextX = a.getX() + a.autoDx;
    double nextY = a.getY() + a.autoDy;

    boolean validNext = canMoveTo(a, nextX, nextY);
    if (!validNext) {
        boolean validRevX = canMoveTo(a, a.getX() - a.autoDx, nextY);
        boolean validRevY = canMoveTo(a, nextX, a.getY() - a.autoDy);

        if (validRevX && !validRevY) {
            a.reverseAutoX();
        } else if (validRevY && !validRevX) {
            a.reverseAutoY();
        } else {
            a.reverseAutoX();
            a.reverseAutoY();
        }
    }
}

a.move(a.autoDx, a.autoDy);

for (Athlete other : athletes) {
    if (other instanceof GymBoss) {
        if (a != other && a.getHitbox().getBoundsInParent().intersects(other.getHitbox().getBoundsInParent())) {
            double e = a.getEnergyLvl();
            if (e > 0.5) a.setEnergyLvl(e - 0.5);
        }
    }
}

```

```

    }
    applyZone(a, detectZone(a));
}

private void handleCollisions(Athlete a) {
    for (Athlete other : athletes) {
        if (other instanceof GymBoss) {
            if (a != other && a.getHitbox().getBoundsInParent().intersects(other.getHitbox().getBoundsInParent())) {
                double e = a.getEnergyLvl();
                if (e > 0.5) a.setEnergyLvl(e - 0.5);
            }
        }
    }
}

private TrainingZone detectZone(Athlete a) {
    return zones.stream()
        .filter(z -> a.getHitbox().getBoundsInParent().intersects(z.getHitbox().getBoundsInParent()))
        .findFirst().orElse(null);
}

private void updateZone(Athlete a) { applyZone(a, detectZone(a)); }

private void applyZone(Athlete a, TrainingZone dz) {
    if (a.getCurrentZone() != dz) {
        if (a.getCurrentZone() != null) a.getCurrentZone().removeAthlete(a);
        if (dz != null) { dz.addAthlete(a); a.setZoneStatus(true, dz.getEmojiImage()); }
        else a.setZoneStatus(false, null);
        a.setCurrentZone(dz);
    }
}

private static boolean isPointInValidArea(double px, double py) {
    for (TrainingZone zone : zones) {
        if (px >= zone.getX() && px <= zone.getX() + zone.getWidth() &&
            py >= zone.getY() && py <= zone.getY() + zone.getHeight()) {
            return true;
        }
    }
    if (px >= CA_X && px <= FD_X && py >= FD_Y + FD_H / 2.0 - COR_W / 2.0 && py <= FD_Y + FD_H / 2.0 + COR_W / 2.0) return true;
    if (px >= CA_X && px <= CA_X + COR_W && py >= FD_Y + FD_H / 2.0 - COR_W / 2.0 && py <= CA_Y) return true;
    if (px >= FD_X + FD_W && px <= WA_X + WA_W && py >= FD_Y + FD_H / 2.0 - COR_W / 2.0 && py <= FD_Y + FD_H / 2.0 + COR_W /
2.0) return true;
    if (px >= WA_X + WA_W - COR_W && px <= WA_X + WA_W && py >= FD_Y + FD_H / 2.0 - COR_W / 2.0 && py <= WA_Y) return true;
    if (px >= FD_X + FD_W / 2.0 - COR_W / 2.0 && px <= FD_X + FD_W / 2.0 + COR_W / 2.0 && py >= FD_Y + FD_H && py <= CA_Y + CA_H
/ 2.0 + COR_W / 2.0) return true;
    if (px >= CA_X + CA_W && px <= FD_X + FD_W / 2.0 + COR_W / 2.0 && py >= CA_Y + CA_H / 2.0 - COR_W / 2.0 && py <= CA_Y +
CA_H / 2.0 + COR_W / 2.0) return true;
    if (px >= FD_X + FD_W / 2.0 - COR_W / 2.0 && px <= WA_X && py >= WA_Y + WA_H / 2.0 - COR_W / 2.0 && py <= WA_Y + WA_H / 2.0
+ COR_W / 2.0) return true;
    return false;
}

```

```

private static boolean canMoveTo(Athlete a, double nextX, double nextY) {
    double hX = nextX;
    double hY = nextY + Athlete.UI_H;
    double hW = Athlete.IMG_W + 4;
    double hH = Athlete.IMG_H;

    if (!isPointInValidArea(hX, hY)) return false;
    if (!isPointInValidArea(hX + hW, hY)) return false;
    if (!isPointInValidArea(hX, hY + hH)) return false;
    if (!isPointInValidArea(hX + hW, hY + hH)) return false;

    if (solidMode) {
        TrainingZone current = a.getCurrentZone();
        if (current != null) {
            if (hX < current.getX() || hX + hW > current.getX() + current.getWidth() ||
                hY < current.getY() || hY + hH > current.getY() + current.getHeight()) {
                return false;
            }
        }
    }
    return true;
}

public static void assignToZoneInitially(Athlete a) {
    zones.stream()
        .filter(z -> a.getHitbox().getBoundsInParent().intersects(z.getHitbox().getBoundsInParent()))
        .findFirst()
        .ifPresent(z -> { z.addAthlete(a); a.setZoneStatus(true, z.getEmojiImage()); a.setCurrentZone(z); });
}
}

```

HelloController.java

```

package gymworld;

import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class HelloController {
    @FXML
    private Label welcomeText;

    @FXML
    protected void onHelloButtonClick() {
        welcomeText.setText("Welcome to JavaFX Application!");
    }
}

```

Athlete.java

```

package gymworld;

import javafx.scene.Group;

```

```

import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.input.MouseButton;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;
import javafx.scene.effect.DropShadow;
import javafx.animation.StrokeTransition;
import javafx.animation.Timeline;
import javafx.util.Duration;
import javafx.animation.SequentialTransition;
import java.io.Serializable;
import java.util.Comparator;
import java.util.Objects;

public class Athlete implements Cloneable, Comparable<Athlete>, Serializable {

    private static final long serialVersionUID = 1L;

    protected static final double IMG_W = 84;
    protected static final double IMG_H = 136.5;
    protected static final double BAR_W = 78 * 1.65;
    protected static final double UI_H = 45;
    protected static final double TOTAL_H = UI_H + IMG_H;

    protected String name;
    protected int age;
    protected double weight;
    protected double energyLvl;
    protected Equipment equipment;
    protected TrainingZone currentZone;

    protected double x;
    protected double y;
    protected boolean isActive;

    protected transient Group visualGroup;
    protected transient ImageView imageView;
    protected transient Text nameText;
    protected transient Text equipmentText;
    protected transient Rectangle textBg;
    protected transient Rectangle energyBarBg;
    protected transient Rectangle energyBar;
    protected transient Rectangle activeBorder;
    protected transient ImageView zoneEmojiView;

    public double autoDx = 0;
    public double autoDy = 0;
    public boolean isAutoMoving = false;

    public Athlete(String name, int age, double weight, double energyLvl, Equipment equipment, double x, double y) {

```

```

this.name = name;
this.age = age;
this.weight = weight;
this.energyLvl = energyLvl;
this.equipment = equipment;
this.x = x;
this.y = y;
this.isActive = false;
this.currentZone = null;
initVisuals();
}

public void initVisuals() {
    visualGroup = new Group();

    activeBorder = new Rectangle(x, y + UI_H, IMG_W + 4, IMG_H);
    activeBorder.setFill(Color.TRANSPARENT);
    activeBorder.setStrokeWidth(2);

    DropShadow glow = new DropShadow();
    glow.setColor(Color.web("#FF00FF"));
    glow.setRadius(12);
    glow.setSpread(0.5);
    activeBorder.setEffect(glow);
    activeBorder.setVisible(false);

    Duration stepDur = Duration.millis(400);
    StrokeTransition t1 = new StrokeTransition(stepDur, activeBorder, Color.RED, Color.ORANGE);
    StrokeTransition t2 = new StrokeTransition(stepDur, activeBorder, Color.ORANGE, Color.YELLOW);
    StrokeTransition t3 = new StrokeTransition(stepDur, activeBorder, Color.YELLOW, Color.LIME);
    StrokeTransition t4 = new StrokeTransition(stepDur, activeBorder, Color.LIME, Color.CYAN);
    StrokeTransition t5 = new StrokeTransition(stepDur, activeBorder, Color.CYAN, Color.BLUE);
    StrokeTransition t6 = new StrokeTransition(stepDur, activeBorder, Color.BLUE, Color.MAGENTA);
    StrokeTransition t7 = new StrokeTransition(stepDur, activeBorder, Color.MAGENTA, Color.RED);
    SequentialTransition rainbow = new SequentialTransition(t1, t2, t3, t4, t5, t6, t7);
    rainbow.setCycleCount(Timeline.INDEFINITE);
    rainbow.play();

    double infoW = (IMG_W + 4) * 1.65;
    double infoX = x + 2 + IMG_W / 2.0 - infoW / 2.0;

    textBg = new Rectangle(infoX, y, infoW, UI_H - 2);
    textBg.setFill(Color.WHITE);

    nameText = new Text(infoX + 6, y + 16, name);
    nameText.setFont(Font.font("Arial", FontWeight.BOLD, 13));
    nameText.setFill(Color.BLACK);

    String eqName = (equipment != null) ? equipment.getName() : "None";
    equipmentText = new Text(infoX + 6, y + 29, "Eq: " + eqName);
    equipmentText.setFont(Font.font("Arial", 10));
    equipmentText.setFill(Color.BLACK);

```

```

double barX = infoX + (infoW - BAR_W) / 2.0;
energyBarBg = new Rectangle(barX, y + UI_H - 9, BAR_W, 7);
energyBarBg.setFill(Color.WHITE);
energyBarBg.setStroke(Color.BLACK);
energyBarBg.setStrokeWidth(0.5);

double barWidth = BAR_W * (energyLvl / 100.0);
energyBar = new Rectangle(barX, y + UI_H - 9, barWidth, 7);

zoneEmojiView = new ImageView();
zoneEmojiView.setX(infoX + infoW - 25);
zoneEmojiView.setY(y + 3);
zoneEmojiView.setFitWidth(21);
zoneEmojiView.setFitHeight(21);
zoneEmojiView.setPreserveRatio(true);
zoneEmojiView.setVisible(false);

imageView = new ImageView(getImagePath());
imageView.setX(x + 2);
imageView.setY(y + UI_H);
imageView.setFitWidth(IMG_W);
imageView.setFitHeight(IMG_H);
imageView.setPreserveRatio(false);

visualGroup.getChildren().addAll(
    imageView, activeBorder, textBg, zoneEmojiView,
    nameText, equipmentText, energyBarBg, energyBar);

drawCustomPrimitives();

visualGroup.setOnMouseClicked(event -> {
    if (event.getButton() == MouseButton.PRIMARY) {
        toggleActive();
    } else if (event.getButton() == MouseButton.SECONDARY) {
        EditDialog.display(this);
    }
    event.consume();
});
}

protected Image getImagePath() {
    return new Image(Objects.requireNonNull(getClass().getResourceAsStream("/gymworld/athlete.png")));
}

protected void drawCustomPrimitives() {
    energyBar.setFill(Color.YELLOW);
}

public void move(double dx, double dy) {
    this.x += dx;
    this.y += dy;

    double infoW = (IMG_W + 4) * 1.65;

```

```

double infoX = this.x + 2 + IMG_W / 2.0 - infoW / 2.0;
double barX = infoX + (infoW - BAR_W) / 2.0;

activeBorder.setX(this.x);
activeBorder.setY(this.y + UI_H);
textBg.setX(infoX);
textBg.setY(this.y);
nameText.setX(infoX + 6);
nameText.setY(this.y + 16);
equipmentText.setX(infoX + 6);
equipmentText.setY(this.y + 29);
energyBarBg.setX(barX);
energyBarBg.setY(this.y + UI_H - 9);
energyBar.setX(barX);
energyBar.setY(this.y + UI_H - 9);
zoneEmojiView.setX(infoX + infoW - 25);
zoneEmojiView.setY(this.y + 3);
imageView.setX(this.x + 2);
imageView.setY(this.y + UI_H);

visualGroup.toFront();
}

public void move(double step) { move(step, step); }

public void move(double distance, String direction) {
    if ("UP".equals(direction)) move(0, -distance);
    else if ("DOWN".equals(direction)) move(0, distance);
    else if ("LEFT".equals(direction)) move(-distance, 0);
    else if ("RIGHT".equals(direction)) move(distance, 0);
}

public void toggleActive() {
    isActive = !isActive;
    activeBorder.setVisible(isActive);
    if (isActive) visualGroup.toFront();
}

public void setZoneStatus(boolean inZone, Image zoneEmoji) {
    if (inZone) {
        textBg.setFill(Color.LIGHTGREEN);
        if (zoneEmoji != null) {
            zoneEmojiView.setImage(zoneEmoji);
            zoneEmojiView.setVisible(true);
        }
    } else {
        textBg.setFill(Color.WHITE);
        zoneEmojiView.setVisible(false);
    }
}

public TrainingZone getCurrentZone() { return currentZone; }
public void setCurrentZone(TrainingZone zone) { this.currentZone = zone; }

```



```

public double getX()          { return x; }
public double getY()          { return y; }
public void removeFromScene()  { GymWorld.rootGroup.getChildren().remove(visualGroup); }
public Group getVisualGroup() { return visualGroup; }
public String getName()       { return name; }
public void setName(String name) {
    this.name = name;
    if (nameText != null) nameText.setText(name);
}
public int getAge()           { return age; }
public void setAge(int age)   { this.age = age; }
public double getWeight()     { return weight; }
public void setWeight(double weight) { this.weight = weight; }
public double getEnergyLvl()  { return energyLvl; }
public void setEnergyLvl(double energyLvl) {
    this.energyLvl = energyLvl;
    if (energyBar != null) energyBar.setWidth(BAR_W * (energyLvl / 100.0));
}
public Equipment getEquipment() { return equipment; }
public void setEquipment(Equipment equipment) {
    this.equipment = equipment;
    if (equipmentText != null)
        equipmentText.setText("Eq: " + (equipment != null ? equipment.getName() : "None"));
}
public String getType()       { return this.getClass().getSimpleName(); }
public void setType(String type) {}
public boolean isActive()     { return isActive; }

@Override
public boolean equals(Object obj) {
    if (obj == null || getClass() != obj.getClass()) return false;
    if (this == obj) return true;
    Athlete other = (Athlete) obj;
    return this.name.equals(other.name) && this.age == other.age && this.weight == other.weight;
}

@Override
public int compareTo(Athlete other) {
    return this.name.compareToIgnoreCase(other.name);
}

@Override
public Object clone() throws CloneNotSupportedException {
    Athlete cloned = (Athlete) super.clone();
    if (this.equipment != null) cloned.setEquipment((Equipment) this.equipment.clone());
    cloned.isActive = false;
    cloned.currentZone = null;
    cloned.x += IMG_W + 10;
    cloned.initVisuals();
    return cloned;
}

public Rectangle getHitbox() { return activeBorder; }

```

```

public static final Comparator<Athlete> NameComparator = (a1, a2) -> a1.getName().compareToIgnoreCase(a2.getName());
public static final Comparator<Athlete> WeightComparator = (a1, a2) -> Double.compare(a1.getWeight(), a2.getWeight());
public static final Comparator<Athlete> EnergyComparator = (a1, a2) -> Double.compare(a1.getEnergyLvl(), a2.getEnergyLvl());

public void reverseAutoX() { this.autoDx = -this.autoDx; }
public void reverseAutoY() { this.autoDy = -this.autoDy; }
}

```

PowerLifter.java

```

package gymworld;

import javafx.scene.image.Image;
import javafx.scene.paint.Color;
import java.io.Serializable;
import java.util.Objects;

public class PowerLifter extends Athlete implements Serializable {

    private static final long serialVersionUID = 2L;

    public PowerLifter(String name, int age, double weight, double energyLvl,
                       Equipment equipment, double x, double y) {
        super(name, age, weight, energyLvl, equipment, x, y);
    }

    @Override
    protected Image getImagePath() {
        return new Image(Objects.requireNonNull(
            getClass().getResourceAsStream("/gymworld/power_lifter.png")));
    }

    @Override
    protected void drawCustomPrimitives() {
        super.drawCustomPrimitives();
        energyBar.setFill(Color.BLUE);
    }
}

```

GymBoss.java

```

package gymworld;

import javafx.scene.image.Image;
import javafx.scene.paint.Color;

import java.io.Serializable;
import java.util.Objects;

public class GymBoss extends PowerLifter implements Serializable {

    private static final long serialVersionUID = 3L;

```

```

public GymBoss(String name, int age, double weight, double energyLvl,
    Equipment equipment, double x, double y) {
    super(name, age, weight, energyLvl, equipment, x, y);
}

@Override
protected Image getImagePath() {
    return new Image(Objects.requireNonNull(
        getClass().getResourceAsStream("/gymworld/gym_boss.png")));
}

@Override
protected void drawCustomPrimitives() {
    super.drawCustomPrimitives();
    energyBar.setFill(Color.RED);
}
}

```

Equipment.java

```

package gymworld;

import java.io.Serializable;

public class Equipment implements Cloneable, Serializable {

    private static final long serialVersionUID = 1L;

    private String name;
    private int value;
    private String unit;

    public Equipment(String name, int value, String unit) {
        this.name = name;
        this.value = value;
        this.unit = unit;
    }

    public String getName()    { return name; }
    public void setName(String name){ this.name = name; }
    public int getValue()      { return value; }
    public void setValue(int value) { this.value = value; }
    public String getUnit()    { return unit; }
    public void setUnit(String unit){ this.unit = unit; }

    @Override
    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }

    @Override
    public String toString() {

```

```

        return name + " (" + value + " " + unit + ")";
    }
}

```

TrainingZone.java

```

package gymworld;

import javafx.scene.Group;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;

import java.io.Serializable;
import java.util.Arrays;
import java.util.Objects;

public class TrainingZone implements Serializable {

    private static final long serialVersionUID = 1L;

    private String name;
    private double x;
    private double y;
    private double width;
    private double height;
    private Athlete[] zoneAthletes;

    private String imagePath;
    private String emojiPath;

    private transient Group visualGroup;
    private transient ImageView background;
    private transient Text nameText;
    private transient Text countText;
    private transient ImageView iconImage;
    private transient Rectangle border;

    public TrainingZone(String name, double x, double y, double width, double height,
                        String imagePath, String emojiPath) {
        this.name = name;
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
        this.imagePath = imagePath;
        this.emojiPath = emojiPath;
        this.zoneAthletes = new Athlete[0];
        initVisuals(imagePath, emojiPath);
    }
}

```

```

}

public void initVisuals() {
    initVisuals(this.imagePath, this.emojiPath);
}

private void initVisuals(String imagePath, String emjPath) {
    visualGroup = new Group();

    Image bgImage = new Image(Objects.requireNonNull(getClass().getResourceAsStream(imagePath)));
    background = new ImageView(bgImage);
    background.setX(x);
    background.setY(y);
    background.setFitWidth(width);
    background.setFitHeight(height);

    border = new Rectangle(x, y, width, height);
    border.setFill(Color.TRANSPARENT);
    border.setStroke(Color.TRANSPARENT);

    Image emojiImg = new Image(Objects.requireNonNull(getClass().getResourceAsStream(emjPath)));
    iconImage = new ImageView(emojiImg);
    iconImage.setX(x + 5);
    iconImage.setY(y + height + 5);
    iconImage.setFitWidth(24);
    iconImage.setFitHeight(24);
    iconImage.setPreserveRatio(true);

    nameText = new Text(x + 35, y + height + 23, name + ":");
    nameText.setFont(Font.font("Arial", FontWeight.BOLD, 16));

    countText = new Text(x + width - 40, y + height + 23, String.valueOf(zoneAthletes.length));
    countText.setFont(Font.font("Arial", FontWeight.BOLD, 18));
    countText.setFill(zoneAthletes.length == 0 ? Color.RED : Color.BLACK);

    visualGroup.getChildren().addAll(background, border, iconImage, nameText, countText);
}

public void addAthlete(Athlete athlete) {
    zoneAthletes = Arrays.copyOf(zoneAthletes, zoneAthletes.length + 1);
    zoneAthletes[zoneAthletes.length - 1] = athlete;
    updateCount();
}

public void removeAthlete(Athlete athlete) {
    int index = -1;
    for (int i = 0; i < zoneAthletes.length; i++) {
        if (zoneAthletes[i].equals(athlete)) { index = i; break; }
    }
    if (index != -1) {
        Athlete[] newArray = new Athlete[zoneAthletes.length - 1];
        for (int i = 0, j = 0; i < zoneAthletes.length; i++) {
            if (i != index) newArray[j++] = zoneAthletes[i];
        }
    }
}

```

```

    }
    zoneAthletes = newArray;
    updateCount();
}
}

private void updateCount() {
    if (countText == null) return;
    countText.setText(String.valueOf(zoneAthletes.length));
    countText.setFill(zoneAthletes.length == 0 ? Color.RED : Color.BLACK);
}

public Group getVisualGroup() { return visualGroup; }
public String getName()      { return name; }
public Image getEmojiImage() { return iconImage.getImage(); }
public Rectangle getHitbox() { return border; }
public double getX()         { return x; }
public double getY()         { return y; }
public double getWidth()     { return width; }
public double getHeight()    { return height; }
}

```

InsertDialog.java

```

package gymworld;

import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;

public class InsertDialog {
    private static Athlete newAthlete = null;

    public static Athlete display() {
        newAthlete = null;
        Stage window = new Stage();
        window.initOwner(GymWorld.mainStage);
        window.initModality(Modality.APPLICATION_MODAL);
        window.setTitle("New Athlete");

        VBox layout = new VBox(10);
        layout.setPadding(new Insets(20));
        layout.getStyleClass().add("dialog-container");

        Label headerLabel = new Label("NEW ATHLETE");
        headerLabel.getStyleClass().add("dialog-header");
    }
}

```

```

Label typeLabel = new Label("Type:");
ComboBox<String> typeCombo = new ComboBox<>();
typeCombo.getItems().addAll("Athlete", "Powerlifter", "Gym Boss");
typeCombo.setValue("Athlete");

Label nameLabel = new Label("Name:");
TextField nameInput = new TextField("New Gym Bro");

Label weightLabel = new Label("Weight (kg):");
TextField weightInput = new TextField("80.0");

CheckBox hasEquipment = new CheckBox("Has Equipment");
hasEquipment.setSelected(true);

Label eqLabel = new Label("Equipment:");
ComboBox<String> eqCombo = new ComboBox<>();
eqCombo.getItems().addAll("Water bottle", "Dumbbell", "Barbell", "Kettlebell", "Jump Rope", "Exercise Band", "Treadmill", "Elliptical machine",
"Stationary bike", "Rowing machine");
eqCombo.setValue("Exercise band");

hasEquipment.setOnAction(e -> eqCombo.setDisable(!hasEquipment.isSelected()));

Label zoneLabel = new Label("Spawn Zone:");
ComboBox<String> zoneCombo = new ComboBox<>();
zoneCombo.getItems().addAll("Front Desk", "Cardio Area", "Weights Area");
zoneCombo.setValue("Front Desk");

Button submitButton = new Button("Create Athlete");
submitButton.setOnAction(e -> {
    String name = nameInput.getText();
    double weight = 80.0;
    try {
        weight = Double.parseDouble(weightInput.getText());
    } catch (NumberFormatException ex) {
        weight = 80.0;
    }

    Equipment eq = null;
    if (hasEquipment.isSelected()) {
        eq = new Equipment(eqCombo.getValue(), 5, "kg");
    }

    double spawnX = 1520;
    double spawnY = 200;
    String selectedZone = zoneCombo.getValue();

    if ("Cardio Area".equals(selectedZone)) {
        spawnX = 300;
        spawnY = 1580;
    } else if ("Weights Area".equals(selectedZone)) {
        spawnX = 2940;
        spawnY = 1580;
    }
});

```

```

    }

    String selectedType = typeCombo.getValue();
    if ("Powerlifter".equals(selectedType)) {
        newAthlete = new PowerLifter(name, 25, weight, 100.0, eq, spawnX, spawnY);
    } else if ("Gym Boss".equals(selectedType)) {
        newAthlete = new GymBoss(name, 45, weight, 100.0, eq, spawnX, spawnY);
    } else {
        newAthlete = new Athlete(name, 20, weight, 100.0, eq, spawnX, spawnY);
    }

    window.close();
});

typeCombo.setMaxWidth(Double.MAX_VALUE);
nameInput.setMaxWidth(Double.MAX_VALUE);
weightInput.setMaxWidth(Double.MAX_VALUE);
eqCombo.setMaxWidth(Double.MAX_VALUE);
zoneCombo.setMaxWidth(Double.MAX_VALUE);
submitButton.setMaxWidth(Double.MAX_VALUE);

layout.getChildren().addAll(headerLabel, typeLabel, typeCombo, nameLabel, nameInput, weightLabel, weightInput, hasEquipment, eqLabel,
eqCombo, zoneLabel, zoneCombo, submitButton);
Scene scene = new Scene(layout, 350, 540);
scene.getStylesheets().add(InsertDialog.class.getResource("/gymworld/style.css").toExternalForm());
window.setScene(scene);
window.showAndWait();

return newAthlete;
}
}

```

EditDialog.java

```

package gymworld;

import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;

public class EditDialog {
    public static void display(Athlete athlete) {
        Stage window = new Stage();
        window.initOwner(GymWorld.mainStage);
        window.initModality(Modality.APPLICATION_MODAL);
        window.setTitle("Edit Athlete");
    }
}

```



```

VBox layout = new VBox(10);
layout.setPadding(new Insets(20));
layout.getStyleClass().add("dialog-container");

Label headerLabel = new Label("EDIT ATHLETE");
headerLabel.getStyleClass().add("dialog-header");

Label typeLabel = new Label("Type:");
ComboBox<String> typeCombo = new ComboBox<>();
typeCombo.getItems().addAll("Athlete", "PowerLifter", "GymBoss");
typeCombo.setValue(athlete.getType());
typeCombo.setDisable(true);

Label nameLabel = new Label("Name:");
TextField nameInput = new TextField(athlete.getName());

Label weightLabel = new Label("Weight (kg):");
TextField weightInput = new TextField(String.valueOf(athlete.getWeight()));

Label energyLabel = new Label("Energy Level (1-100):");
TextField energyInput = new TextField(String.valueOf(athlete.getEnergyLvl()));

CheckBox hasEquipment = new CheckBox("Has Equipment");
hasEquipment.setSelected(athlete.getEquipment() != null);

Label eqLabel = new Label("Equipment:");
ComboBox<String> eqCombo = new ComboBox<>();
eqCombo.getItems().addAll("Water bottle", "Dumbbell", "Barbell", "Kettlebell", "Jump Rope", "Exercise Band", "Treadmill", "Elliptical machine",
"Stationary bike", "Rowing machine");

if (athlete.getEquipment() != null) {
    eqCombo.setValue(athlete.getEquipment().getName());
} else {
    eqCombo.setValue("Exercise Band");
    eqCombo.setDisable(true);
}

hasEquipment.setOnAction(e -> eqCombo.setDisable(!hasEquipment.isSelected()));

Button submitButton = new Button("Save Changes");
submitButton.setOnAction(e -> {
    athlete.setName(nameInput.getText());

    try {
        athlete.setWeight(Double.parseDouble(weightInput.getText()));
    } catch (NumberFormatException ex) {
        athlete.setWeight(80.0);
    }

    try {
        double energy = Double.parseDouble(energyInput.getText());
        if (energy < 1.0) energy = 1.0;
    }
}

```

```

        if (energy > 100.0) energy = 100.0;
        athlete.setEnergyLvl(energy);
    } catch (NumberFormatException ex) {
    }

    if (hasEquipment.isSelected()) {
        athlete.setEquipment(new Equipment(eqCombo.getValue(), 5, "kg"));
    } else {
        athlete.setEquipment(null);
    }

    window.close();
});

typeCombo.setMaxWidth(Double.MAX_VALUE);
nameInput.setMaxWidth(Double.MAX_VALUE);
weightInput.setMaxWidth(Double.MAX_VALUE);
energyInput.setMaxWidth(Double.MAX_VALUE);
eqCombo.setMaxWidth(Double.MAX_VALUE);
submitButton.setMaxWidth(Double.MAX_VALUE);

layout.getChildren().addAll(headerLabel, typeLabel, typeCombo, nameLabel, nameInput, weightLabel, weightInput, energyLabel, energyInput,
hasEquipment, eqLabel, eqCombo, submitButton);

Scene scene = new Scene(layout, 350, 580);
scene.getStylesheets().add(EditDialog.class.getResource("/gymworld/style.css").toExternalForm());
window.setScene(scene);
window.showAndWait();
}
}

```

SearchDialog.java

```

package gymworld;

import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;

import java.util.Optional;

public class SearchDialog {

    public static void display() {
        Stage window = new Stage();
        window.initOwner(GymWorld.mainStage);
        window.initModality(Modality.APPLICATION_MODAL);
    }
}

```

```

window.setTitle("Search Athlete");

VBox layout = new VBox(10);
layout.setPadding(new Insets(20));
layout.getStyleClass().add("dialog-container");

Label headerLabel = new Label("SEARCH ATHLETE");
headerLabel.getStyleClass().add("dialog-header");

Label nameLabel = new Label("Enter Athlete Name:");
TextField nameInput = new TextField();

Button searchButton = new Button("Search");
searchButton.setOnAction(e -> {
    String targetName = nameInput.getText().trim().toLowerCase();

    Optional<Athlete> found = GymWorld.athletes.stream()
        .filter(a -> a.getName().toLowerCase().contains(targetName))
        .findFirst();

    window.close();

    if (found.isPresent()) {
        Athlete a = found.get();
        String zoneName = (a.getCurrentZone() != null)
            ? a.getCurrentZone().getName()
            : "None (Free Object)";

        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.getDialogPane().getStylesheets().add(SearchDialog.class.getResource("/gymworld/style.css").toExternalForm());
        alert.setTitle("Search Result");
        alert.setHeaderText("Athlete Found!");
        alert.setContentText(
            "Name: " + a.getName() + "\n" +
            "Class Type: " + a.getType() + "\n" +
            "Location: X = " + (int) a.getX() + ", Y = " + (int) a.getY() + "\n" +
            "Macro-object (Zone): " + zoneName);
        alert.showAndWait();
    } else {
        Alert alert = new Alert(Alert.AlertType.WARNING);
        alert.getDialogPane().getStylesheets().add(SearchDialog.class.getResource("/gymworld/style.css").toExternalForm());
        alert.setTitle("Search Result");
        alert.setHeaderText("Not Found");
        alert.setContentText("Athlete with name " + targetName + " was not found.");
        alert.showAndWait();
    }
});

nameInput.setMaxWidth(Double.MAX_VALUE);
searchButton.setMaxWidth(Double.MAX_VALUE);

layout.getChildren().addAll(headerLabel, nameLabel, nameInput, searchButton);
Scene scene = new Scene(layout, 350, 200);

```

```

        scene.getStylesheets().add(SearchDialog.class.getResource("/gymworld/style.css").toExternalForm());
        window.setScene(scene);
        window.showAndWait();
    }
}

```

FilterDialog.java

```

package gymworld;

import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.TableView;
import javafx.scene.control.TableColumn;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;

import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

public class FilterDialog {

    public static void display() {
        Stage window = new Stage();
        window.initOwner(GymWorld.mainStage);
        window.initModality(Modality.APPLICATION_MODAL);
        window.setTitle("Filter and Sort Athletes");

        VBox layout = new VBox(10);
        layout.setPadding(new Insets(20));
        layout.getStyleClass().add("dialog-container");

        Label headerLabel = new Label("GYM WORLD QUERIES");
        headerLabel.getStyleClass().add("dialog-header");

        Label filterLabel = new Label("Query A — Filter by Zone:");
        ComboBox<String> filterCombo = new ComboBox<>();
        GymWorld.zones.stream()
            .map(TrainingZone::getName)
            .forEach(filterCombo.getItems()::add);
        filterCombo.getItems().add("Free Objects (None)");
        filterCombo.setValue(filterCombo.getItems().get(0));
    }
}

```

```

Label sortLabel = new Label("Sort By:");
ComboBox<String> sortCombo = new ComboBox<>();
sortCombo.getItems().addAll("Name", "Weight", "Energy Level");
sortCombo.setValue("Name");

Button queryAButton = new Button("Run Query A (filter + sort)");
queryAButton.setOnAction(e -> {
    String selectedFilter = filterCombo.getValue();
    String selectedSort = sortCombo.getValue();

    List<Athlete> result = GymWorld.athletes.stream()
        .filter(a -> {
            if ("Free Objects (None)".equals(selectedFilter)) {
                return a.getCurrentZone() == null;
            } else {
                return a.getCurrentZone() != null
                    && a.getCurrentZone().getName().equals(selectedFilter);
            }
        })
        .sorted(resolveSortComparator(selectedSort))
        .collect(Collectors.toList());

    StringBuilder sb = new StringBuilder();
    result.forEach(a -> sb
        .append("- ").append(a.getName())
        .append(" [").append(a.getType()).append("] | ")
        .append("Weight: ").append(a.getWeight()).append(" kg | ")
        .append("Energy: ").append(String.format("%.1f", a.getEnergyLvl())).append("\n"));

    window.close();
    showInfo("Query A Result",
        "Filter: " + selectedFilter + " | Sorted by: " + selectedSort,
        result.isEmpty()
            ? "No athletes found in this category."
            : "Found " + result.size() + " athlete(s):\n\n" + sb);
});

Button queryBButton = new Button("Run Query B (energy > 50%)");
queryBButton.setOnAction(e -> {
    long count = GymWorld.athletes.stream()
        .filter(a -> a.getEnergyLvl() > 50.0)
        .count();

    String names = GymWorld.athletes.stream()
        .filter(a -> a.getEnergyLvl() > 50.0)
        .sorted(Comparator.comparingDouble(Athlete::getEnergyLvl).reversed())
        .map(a -> a.getName() + " (" + String.format("%.1f", a.getEnergyLvl()) + "%)")
        .collect(Collectors.joining(", "));

    window.close();
    showInfo("Query B — High Energy Athletes",
        "Athletes with energy level > 50%",
        "Count: " + count + "\n\n" + (names.isEmpty() ? "None found." : names));
});

```

```

});

Button queryCButton = new Button("Run Query C (right half of world, x >= 1920)");
queryCButton.setOnAction(e -> {
    long count = GymWorld.athletes.stream()
        .filter(a -> a.getX() >= 1920)
        .count();

    String names = GymWorld.athletes.stream()
        .filter(a -> a.getX() >= 1920)
        .sorted(Athlete.NameComparator)
        .map(a -> a.getName() + " (x=" + (int) a.getX() + ")")
        .collect(Collectors.joining(", "));

    window.close();
    showInfo("Query C — Right Half of World",
        "Athletes with X coordinate >= 960",
        "Count: " + count + "\n\n" + (names.isEmpty() ? "None found." : names));
});

filterCombo.setMaxWidth(Double.MAX_VALUE);
sortCombo.setMaxWidth(Double.MAX_VALUE);
queryAButton.setMaxWidth(Double.MAX_VALUE);
queryBButton.setMaxWidth(Double.MAX_VALUE);
queryCButton.setMaxWidth(Double.MAX_VALUE);

Label tableLabel = new Label("ALL ACTIVE ATHLETES IN THE WORLD:");
tableLabel.getStyleClass().add("section-label");

TableView<Athlete> tableView = new TableView<>();
tableView.setPrefHeight(220);

TableColumn<Athlete, String> nameCol = new TableColumn<>("Name");
nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
nameCol.setPrefWidth(130);

TableColumn<Athlete, String> typeCol = new TableColumn<>("Type");
typeCol.setCellValueFactory(new PropertyValueFactory<>("type"));
typeCol.setPrefWidth(100);

TableColumn<Athlete, Double> weightCol = new TableColumn<>("Weight (kg)");
weightCol.setCellValueFactory(new PropertyValueFactory<>("weight"));
weightCol.setPrefWidth(90);

TableColumn<Athlete, Double> energyCol = new TableColumn<>("Energy (%)");
energyCol.setCellValueFactory(new PropertyValueFactory<>("energyLvl"));
energyCol.setPrefWidth(90);

TableColumn<Athlete, String> eqCol = new TableColumn<>("Equipment");
eqCol.setCellValueFactory(cellData -> {
    Equipment eq = cellData.getValue().getEquipment();
    return new SimpleStringProperty(eq != null ? eq.getName() : "None");
});

```

```

eqCol.setPrefWidth(125);

TableColumn<Athlete, String> zoneCol = new TableColumn<>("Current Zone");
zoneCol.setCellValueFactory(cellData -> {
    TrainingZone zone = cellData.getValue().getCurrentZone();
    return new SimpleStringProperty(zone != null ? zone.getName() : "None (Free)");
});
zoneCol.setPrefWidth(125);

tableView.getColumns().addAll(nameCol, typeCol, weightCol, energyCol, eqCol, zoneCol);

ObservableList<Athlete> data = FXCollections.observableArrayList(GymWorld.athletes);
tableView.setItems(data);

layout.getChildren().addAll(
    headerLabel,
    filterLabel, filterCombo,
    sortLabel, sortCombo,
    queryAButton, queryBButton, queryCButton,
    tableLabel, tableView
);

Scene scene = new Scene(layout, 720, 640);
scene.getStylesheets().add(FilterDialog.class.getResource("/gymworld/style.css").toExternalForm());
window.setScene(scene);
window.showAndWait();
}

private static Comparator<Athlete> resolveSortComparator(String criterion) {
    return switch (criterion) {
        case "Weight"    -> Athlete.WeightComparator;
        case "Energy Level" -> Athlete.EnergyComparator;
        default          -> Athlete.NameComparator;
    };
}

private static void showInfo(String title, String header, String content) {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.getDialogPane().getStylesheets().add(FilterDialog.class.getResource("/gymworld/style.css").toExternalForm());
    alert.setTitle(title);
    alert.setHeaderText(header);
    alert.setContentText(content);
    alert.showAndWait();
}
}

```

Minimap.java

```

package gymworld;

import javafx.scene.Group;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;

```

```

import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;

public class Minimap {

    private static final double MAP_W = 240;
    private static final double MAP_H = 135;
    private static final double MAP_X = 10;
    private static final double MAP_Y = 10;

    private double scaleX;
    private double scaleY;

    private double viewportW;
    private double viewportH;

    private final Group root;
    private final Rectangle bg;
    private final Rectangle viewportRect;
    private final Group corridorLayer = new Group();
    private final Group zoneLayer = new Group();
    private final Group athleteLayer = new Group();
    private final java.util.Map<Athlete, Rectangle> dotMap = new java.util.HashMap<>();

    public Minimap(double viewportW, double viewportH) {
        this.viewportW = viewportW;
        this.viewportH = viewportH;
        this.scaleX = MAP_W / GymWorld.WORLD_W;
        this.scaleY = MAP_H / GymWorld.WORLD_H;

        root = new Group();

        bg = new Rectangle(MAP_X, MAP_Y, MAP_W, MAP_H);
        bg.setFill(Color.color(0.1, 0.1, 0.1, 0.75));
        bg.setStroke(Color.GRAY);
        bg.setStrokeWidth(1);

        viewportRect = new Rectangle(MAP_X, MAP_Y,
            viewportW * scaleX,
            viewportH * scaleY);
        viewportRect.setFill(Color.color(1, 1, 1, 0.25));
        viewportRect.setStroke(Color.WHITE);
        viewportRect.setStrokeWidth(1);

        Text label = new Text(MAP_X + 3, MAP_Y + MAP_H + 12, "Minimap [click to move camera]");
        label.setFont(Font.font("Arial", FontWeight.NORMAL, 9));
        label.setFill(Color.BLACK);

        root.getChildren().addAll(bg, corridorLayer, zoneLayer, athleteLayer, viewportRect, label);

        root.setOnMouseClicked(event -> {
            double relX = event.getX() - MAP_X;

```



```

double relY = event.getY() - MAP_Y;

if (relX < 0 || relY < 0 || relX > MAP_W || relY > MAP_H) return;

double worldX = relX / scaleX;
double worldY = relY / scaleY;

double newCamX = worldX - viewportW / 2.0;
double newCamY = worldY - viewportH / 2.0;

newCamX = Math.max(0, Math.min(GymWorld.WORLD_W - viewportW, newCamX));
newCamY = Math.max(0, Math.min(GymWorld.WORLD_H - viewportH, newCamY));

GymWorld.camX = newCamX;
GymWorld.camY = newCamY;
GymWorld.applyCamera();
});
}

public void resize(double newViewportW, double newViewportH) {
    this.viewportW = newViewportW;
    this.viewportH = newViewportH;
    this.scaleX = MAP_W / GymWorld.WORLD_W;
    this.scaleY = MAP_H / GymWorld.WORLD_H;
    viewportRect.setWidth(newViewportW * scaleX);
    viewportRect.setHeight(newViewportH * scaleY);
    zoneLayer.getChildren().clear();
    corridorLayer.getChildren().clear();
}

public void update(double cameraTransX, double cameraTransY) {
    if (corridorLayer.getChildren().isEmpty() && !GymWorld.corridors.isEmpty()) {
        for (Rectangle cor : GymWorld.corridors) {
            Rectangle r = new Rectangle(
                MAP_X + cor.getX() * scaleX,
                MAP_Y + cor.getY() * scaleY,
                cor.getWidth() * scaleX,
                cor.getHeight() * scaleY);
            r.setFill(cor.getFill());
            corridorLayer.getChildren().add(r);
        }
    }

    if (zoneLayer.getChildren().size() != GymWorld.zones.size()) {
        zoneLayer.getChildren().clear();
        for (TrainingZone zone : GymWorld.zones) {
            Rectangle r = new Rectangle(
                MAP_X + zone.getX() * scaleX,
                MAP_Y + zone.getY() * scaleY,
                zone.getWidth() * scaleX,
                zone.getHeight() * scaleY);
            if (zone.getName().equalsIgnoreCase("Front Desk")) {
                r.setFill(Color.color(0.9, 0.2, 0.2, 0.5));
            }
        }
    }
}

```

```

        r.setStroke(Color.RED);
    } else if (zone.getName().equalsIgnoreCase("Cardio Area")) {
        r.setFill(Color.color(0.9, 0.9, 0.2, 0.5));
        r.setStroke(Color.YELLOW);
    } else {
        r.setFill(Color.color(0.2, 0.5, 0.9, 0.5));
        r.setStroke(Color.CORNFLOWERBLUE);
    }
    r.setStrokeWidth(0.5);
    zoneLayer.getChildren().add(r);
}
}

dotMap.keySet().removeIf(a -> {
    if (!GymWorld.athletes.contains(a)) {
        athleteLayer.getChildren().remove(dotMap.get(a));
        return true;
    }
    return false;
});

for (Athlete a : GymWorld.athletes) {
    Rectangle dot = dotMap.get(a);
    if (dot == null) {
        dot = new Rectangle(0, 0, 4, 4);
        if (a instanceof GymBoss) dot.setFill(Color.RED);
        else if (a instanceof PowerLifter) dot.setFill(Color.BLUE);
        else dot.setFill(Color.YELLOW);
        dot.setStroke(Color.BLACK);
        dot.setStrokeWidth(0.5);
        dotMap.put(a, dot);
        athleteLayer.getChildren().add(dot);
    }
    dot.setX(MAP_X + a.getX() * scaleX);
    dot.setY(MAP_Y + a.getY() * scaleY);
}

viewportRect.setX(MAP_X + (-cameraTransX) * scaleX);
viewportRect.setY(MAP_Y + (-cameraTransY) * scaleY);
}

public void reset() {
    zoneLayer.getChildren().clear();
    corridorLayer.getChildren().clear();
    athleteLayer.getChildren().clear();
    dotMap.clear();
}

public Group getRoot() { return root; }
}

```

SerializationManager.java

```

package gymworld;

import javafx.stage.FileChooser;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class SerializationManager {

    public static void save() {
        FileChooser chooser = new FileChooser();
        chooser.setTitle("Save GymWorld State");
        chooser.getExtensionFilters().add(
            new FileChooser.ExtensionFilter("GymWorld Save File (*.gym)", "*.gym"));
        chooser.setInitialFileName("gymworld_save.gym");

        File file = chooser.showSaveDialog(GymWorld.mainStage);
        if (file == null) return;

        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(file))) {

            oos.writeObject(new ArrayList<>(GymWorld.zones));
            oos.writeObject(new ArrayList<>(GymWorld.athletes));

            showInfo("Save Successful",
                "World state saved to:\n" + file.getAbsolutePath());

        } catch (IOException ex) {
            showError("Save Failed", ex.getMessage());
        }
    }

    public static void load() {
        FileChooser chooser = new FileChooser();
        chooser.setTitle("Load GymWorld State");
        chooser.getExtensionFilters().add(
            new FileChooser.ExtensionFilter("GymWorld Save File (*.gym)", "*.gym"));

        File file = chooser.showOpenDialog(GymWorld.mainStage);
        if (file == null) return;

        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file))) {

            List<TrainingZone> loadedZones = (List<TrainingZone>) ois.readObject();
            List<Athlete> loadedAthletes = (List<Athlete>) ois.readObject();

            for (Athlete a : GymWorld.athletes) {
                GymWorld.rootGroup.getChildren().remove(a.getVisualGroup());
            }

            for (TrainingZone z : GymWorld.zones) {

```

```

        GymWorld.rootGroup.getChildren().remove(z.getVisualGroup());
    }
    GymWorld.athletes.clear();
    GymWorld.zones.clear();
    if (GymWorld.minimap != null) {
        GymWorld.minimap.reset();
    }

    for (TrainingZone zone : loadedZones) {
        zone.initVisuals();
        GymWorld.zones.add(zone);
        GymWorld.rootGroup.getChildren().add(zone.getVisualGroup());
    }

    for (Athlete athlete : loadedAthletes) {
        athlete.initVisuals();

        TrainingZone zone = athlete.getCurrentZone();
        if (zone != null) {
            athlete.setZoneStatus(true, zone.getEmojiImage());
        }

        GymWorld.athletes.add(athlete);
        GymWorld.rootGroup.getChildren().add(athlete.getVisualGroup());
    }

    showInfo("Load Successful",
            "World state loaded from:\n" + file.getAbsolutePath());

} catch (IOException | ClassNotFoundException ex) {
    showError("Load Failed", ex.getMessage());
}
}

private static void showInfo(String title, String message) {
    javafx.scene.control.Alert alert =
        new javafx.scene.control.Alert(javafx.scene.control.Alert.AlertType.INFORMATION);
    alert.setTitle(title);
    alert.setHeaderText(null);
    alert.setContentText(message);
    alert.showAndWait();
}

private static void showError(String title, String message) {
    javafx.scene.control.Alert alert =
        new javafx.scene.control.Alert(javafx.scene.control.Alert.AlertType.ERROR);
    alert.setTitle(title);
    alert.setHeaderText("An error occurred");
    alert.setContentText(message);
    alert.showAndWait();
}
}

```